

P R O M P T E N G I N E E R I N G M E I S T E R N

B A N D 7

Prompting-fuer-Entwickler

Code, APIs und Automatisierung

Belkis Aslani

2026

Prompt Engineering Meistern

Band 7: Prompting-fuer-Entwickler – Code, APIs und Automatisierung

© 2026 Belkis Aslani. Alle Rechte vorbehalten.

1. Auflage, März 2026

Dieses Werk ist urheberrechtlich geschützt. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen und die Einspeicherung und Verarbeitung in elektronischen Systemen.

Die in diesem Buch genannten Produkt- und Firmennamen sind Marken der jeweiligen Eigentümer.

Satz und Layout: Eigensatz des Autors

Umschlaggestaltung: Belkis Aslani

Inhaltsverzeichnis

Vorwort

- 1** Code-Generierung mit KI – Von der Funktion zum System

- 2** Agentic Coding Tools – Die neue Generation der Entwicklerwerkzeuge

- 3** Die LLM-APIs – Dein Zugang zur KI-Power

- 4** Programmatisches Prompting – Prompts als Code

- 5** RAG – Retrieval Augmented Generation

- 6** Tool Use und Function Calling – KI greift in die echte Welt

- 7** Agentische Systeme – KI, die handelt

- 8** Fine-Tuning vs. Prompting – Die Entscheidungsmatrix

- 9** Context Engineering – Die Evolution des Prompt Engineering

- 10** Zusammenfassung und Ausblick

Vorwort

Sechs Bände lang warst du Endnutzer. Du hast in Chat-Fenster getippt, Ergebnisse gelesen, iteriert. Gut so – das ist die Grundlage.

Aber jetzt bist du bereit für den nächsten Schritt: KI nicht nur nutzen, sondern einbauen. In deine Software. In deine Workflows. In deine Produkte.

Was diesen Band anders macht

In den Bänden 1 bis 6 ging es darum, wie du mit KI *kommunizierst*. In Band 7 geht es darum, wie du KI *programmierst*.

Das ist ein fundamentaler Unterschied:

- Du schreibst nicht mehr einzelne Prompts – du baust Systeme, die tausende Prompts automatisch generieren und verarbeiten.
- Du copy-pastest keine Antworten mehr – du verarbeitest strukturierte API-Responses in deinem Code.
- Du wartest nicht mehr auf Ergebnisse – du baust Pipelines, die asynchron, parallel und skalierbar arbeiten.
- Du promptest nicht mehr allein – du baust Agenten, die Tools nutzen, Entscheidungen treffen und selbstständig handeln.

Willkommen in der Welt des programmatischen Promptings.

Was du brauchst

Technische Voraussetzungen für diesen Band:

- **Grundlegende Programmierkenntnisse** – Python oder JavaScript. Du musst kein Senior Developer sein, aber du solltest wissen, was eine Funktion, eine Variable und ein API-Call ist.
- **Ein Terminal/Kommandozeile** – Du wirst Befehle ausführen.
- **API-Zugang** – Mindestens einen API-Key von Anthropic (Claude), OpenAI oder Google (Gemini). Kostenlose Tiers reichen für die Übungen.
- **Eine Entwicklungsumgebung** – VS Code, Cursor, oder ein beliebiger Editor.
- **Neugier** – Weil die Landschaft sich schnell bewegt und du experimentieren musst.

Wenn du nicht programmierst: Lies trotzdem die Kapitel über Code-Generierung (Kapitel 1-2) und das Konzept-Kapitel über Context Engineering (Kapitel 9). Der Rest ist für Entwickler.

Was dich erwartet

Code-Generierung – Nicht “Schreib mir eine Funktion”, sondern: Wie du KI für Architekturentscheidungen, Code-Reviews, Refactoring und Test-Generierung nutzt. Von der Einzelfunktion zum Gesamtsystem.

Agentic Coding Tools – Cursor, GitHub Copilot, Claude Code, Windsurf, Cline, Aider. Was können sie? Wie unterscheiden sie sich? Und: Wie promptest du innerhalb dieser Tools optimal?

Die LLM-APIs – Anthropic, OpenAI, Google, Open Source. Preise, Modelle, Features. Structured Output, Vision, Streaming, Caching. Alles, was du als Entwickler wissen musst.

Programmatisches Prompting – Prompt-Templates, Variablen, Antwortverarbeitung, Fehlerbehandlung, Retry-Logik. Mit echtem Code in Python und TypeScript.

RAG (Retrieval Augmented Generation) – Deine eigenen Daten in die KI bringen. Embeddings, Vektor-Datenbanken, Chunking-Strategien. Das Fundament jeder ernsthaften KI-Anwendung.

Tool Use und Function Calling – KI, die nicht nur Text generiert, sondern Funktionen aufruft: Datenbanken abfragen, APIs ansprechen, Dateien bearbeiten. Die Brücke zwischen Text und Aktion.

Agentische Systeme – KI-Agenten, die planen, handeln und iterieren. MCP (Model Context Protocol), Multi-Step-Agents, Orchestrierung. Der aktuellste und spannendste Bereich.

Fine-Tuning vs. Prompting – Wann reicht gutes Prompting? Wann lohnt sich Fine-Tuning? Die Entscheidungsmatrix mit konkreten Kosten-Nutzen-Analysen.

Context Engineering – Der Nachfolger von Prompt Engineering. Wie du den gesamten Kontext eines LLM-Aufrufs designst – nicht nur den Prompt, sondern System-Prompts, Tool-Definitionen, Retrieval-Ergebnisse, Konversationshistorie und Caching.

Die KI-Entwickler-Landschaft 2026

Bevor wir einsteigen, ein Überblick über den Stand der Dinge:

Bereich	Stand 2024	Stand 2026
Code-Assisten- ten	GitHub Copilot do- miniert	Dutzende Tools, agentic by default
APIs	Text rein, Text raus	Multimodal, Tool Use, Structu- red Output
RAG	Experimentell, kom- plex	Standard-Pattern, viele Frame- works
Agenten	Demo-Stage, fragil	Produktionsreif für definierte Workflows
Fine-Tuning	Teuer, komplex	Günstiger, aber Prompting oft ausreichend
Context Engi- neering	Buzzword	Etablierte Disziplin
Open Source	Llama 2, Mixtral	Llama 4, DeepSeek-V3, Qwen 3

Die Entwicklung in den letzten zwei Jahren war exponentiell. Was 2024 als Research-Paper existierte, ist 2026 ein npm-Paket. Was 2024 ein Startup war, ist 2026 ein Feature von VS Code.

Dieses Buch gibt dir das Wissen, um in dieser Landschaft nicht nur mitzuschwimmen, sondern zu navigieren.

Los geht's. Wir bauen.

Belkis Aslani, März 2026

Kapitel 1: Code-Generierung mit KI – Von der Funktion zum System

“Schreib mir eine Python-Funktion, die prüft, ob eine Zahl eine Primzahl ist.” Das kann jeder. Und es ist ungefähr so nützlich wie “Schreib mir ein Gedicht über Liebe” – du bekommst ein generisches Ergebnis, das meistens funktioniert, aber selten genau das ist, was du brauchst.

In diesem Kapitel lernst du, wie du KI für professionelle Software-Entwicklung nutzt – von der Einzelfunktion bis zur Systemarchitektur.

Warum die meisten Code-Prompts schlecht sind

Problem 1: Zu wenig Kontext

SCHLECHT:

"Schreib eine Login-Funktion."

GUT:

"Schreib eine Login-Funktion für eine Express.js-API.
Tech-Stack: Node.js 22, TypeScript 5.4, PostgreSQL mit Prisma ORM.

Auth: JWT mit Access + Refresh Token (RS256).

Passwort-Hashing: bcrypt, 12 Rounds.

Rate Limiting: Max. 5 Versuche pro 15 Min pro IP.

Input-Validierung: Zod-Schema.

Error Handling: Custom AppError-Klasse mit HTTP-Statuscodes.

Logging: Pino-Logger, keine sensiblen Daten loggen."

Problem 2: Kein existierender Codebase-Kontext

KI weiß nicht, wie dein Projekt aufgebaut ist. Ohne Kontext schreibt sie Code, der nicht in dein Projekt passt.

Hier ist mein Projekt-Kontext:

ARCHITEKTUR: Hexagonal Architecture (Ports & Adapters)

ORDNERSTRUKTUR:

```
src/  
  domain/      # Business-Logik, Entities, Value Objects  
  application/ # Use Cases, Ports (Interfaces)  
  infrastructure/ # Adapter (DB, API, etc.)  
  api/         # REST Controller, DTOs
```

KONVENTIONEN:

- Error Handling über Result-Type (kein throw)
- Dependency Injection via Constructor
- Alle DB-Zugriffe über Repository-Interfaces
- Tests: Vitest, AAA-Pattern, keine Mocks für Domain-Logik

Schreibe [AUFGABE] so, dass sie in diese Architektur passt.

Problem 3: Keine Qualitätsanforderungen

SCHLECHT:

"Schreib eine Funktion zum Sortieren."

GUT:

"Schreib eine Sortierfunktion mit folgenden Anforderungen:

- TypeScript, generisch (funktioniert mit jedem Typ)
- Stabile Sortierung
- $O(n \log n)$ im Average Case
- Immutable: Gibt neues Array zurück, verändert das Original nicht
- Vergleichsfunktion als Parameter (wie `Array.sort`)
- JSDoc-Dokumentation
- Edge Cases: Leeres Array, ein Element, bereits sortiert
- Unit Tests mit Vitest (min. 5 Testfälle)"

Der professionelle Code-Prompt

Mein Framework für Code-Generierung:

AUFGABE: [Was soll gebaut werden?]

KONTEXT: [Wo wird es eingebaut? Existierender Code?]

TECH-STACK: [Sprache, Framework, Versionen]

ARCHITEKTUR: [Patterns, Strukturen, Konventionen]

ANFORDERUNGEN:

- Funktional: [Was muss es tun?]
- Nicht-funktional: [Performance, Sicherheit, Skalierbarkeit]
- Edge Cases: [Was kann schiefgehen?]

QUALITÄT:

- Error Handling: [Strategie]
- Logging: [Was loggen, was nicht?]
- Tests: [Test-Framework, Testfälle]
- Dokumentation: [JSDoc/Docstring/README]

EINSCHRÄNKUNGEN:

- [Was NICHT verwenden: bestimmte Libraries, Patterns]
- [Max. Dateigröße, Komplexität]
- [Kompatibilität: Browser, Node-Versionen]

Code-Generierung nach Aufgabentyp

Neue Features implementieren

Implementiere folgendes Feature:

FEATURE: [Beschreibung]

USER STORY: "Als [Rolle] möchte ich [Aktion],
damit [Nutzen]."

AKZEPTANZKRITERIEN:

1. [Kriterium 1]
2. [Kriterium 2]
3. [Kriterium 3]

EXISTIERENDER CODE:

""[Relevante Code-Abschnitte einfügen]""

DATENMODELL: [Welche Daten sind beteiligt?]

IMPLEMENTIERE:

1. Datenbank-Migration (wenn nötig)
2. Backend-Logik (Service + Repository)
3. API-Endpoint (Controller + Validierung)
4. Tests (Unit + Integration)

Keine Frontend-Änderungen in diesem PR.

Bugs fixen

BUG: [Beschreibung des Problems]
ERWARTETES VERHALTEN: [Was sollte passieren?]
TATSÄCHLICHES VERHALTEN: [Was passiert stattdessen?]
REPRODUKTION: [Schritte zum Reproduzieren]

FEHLERMELDUNG:
""[Error-Log einfügen]""

RELEVANTER CODE:
""[Code-Abschnitt einfügen]""

AUFGABE:

1. Identifiziere die Ursache (Root Cause Analysis)
2. Schlage einen Fix vor (erkläre warum)
3. Implementiere den Fix
4. Schreibe einen Regressions-Test, der den Bug abdeckt
5. Prüfe: Kann der Fix andere Stellen beeinflussen?

Refactoring

Refactore folgenden Code:

""[Code einfügen]""

ZIEL: [z.B. Lesbarkeit, Performance, Testbarkeit,
Separation of Concerns, DRY]

REGELN:

- Verhalten darf sich NICHT ändern (pure Refactoring)
- Existierende Tests müssen weiterhin grün sein
- Keine neuen Dependencies einführen
- Commits in logische Schritte aufteilen

ERKLÄRE für jede Änderung:

- Was wurde geändert?
- Warum?
- Welches Design-Pattern wurde angewandt?

Code Reviews

Reviewe folgenden Code:

```
""[Code einfügen / PR-Diff]""
```

PRÜFE AUF:

1. KORREKTHEIT: Tut der Code, was er soll?
2. SICHERHEIT: SQL Injection, XSS, Auth-Lücken?
3. PERFORMANCE: N+1 Queries? Unnötige Berechnungen?
4. LESBARKEIT: Verständliche Variablennamen? Kommentare nötig?
5. FEHLERBEHANDLUNG: Werden Fehler abgefangen und sinnvoll behandelt?
6. TESTS: Ausreichend abgedeckt? Edge Cases?
7. KONVENTIONEN: Passt es zum Rest des Projekts?

FORMAT:

Für jedes Finding:

- Datei + Zeile
- Schwere:  Blocker /  Verbesserung /  Nitpick
- Problem
- Vorgeschlagener Fix

Test-Generierung

Schreibe Tests für folgenden Code:

```
""[Code einfügen]""
```

TEST-FRAMEWORK: [Jest/Vitest/pytest/Go testing/...]

TEST-ARTEN:

- Unit Tests (isoliert, gemockte Dependencies)
- Integration Tests (mit DB/API)
- Edge Case Tests

NAMING: describe/it-Pattern (BDD-Style)

PATTERN: Arrange-Act-Assert (AAA)

ABDECKUNG:

- Happy Path (normaler Ablauf)
- Error Cases (was kann schiefgehen?)
- Boundary Values (Grenzwerte)
- Null/Undefined/Empty
- Concurrent Access (wenn relevant)

Mindestens [X] Testfälle. Ziel: >90% Branch Coverage.

Architektur-Prompts

System Design

Designe die Architektur für folgendes System:

SYSTEM: [Beschreibung]

ANFORDERUNGEN:

- Funktional: [Was muss es können?]
- Nicht-funktional:
 - Erwartete Last: [Requests/Sekunde]
 - Verfügbarkeit: [99.9%? 99.99%?]
 - Latenz: [P95 < Xms]
 - Datenmenge: [Wie viel Daten?]
 - Compliance: [DSGVO? PCI-DSS?]

ERSTELLE:

1. High-Level-Architektur (Komponenten + Interaktion)
2. Datenfluss (Wie fließen Daten durch das System?)
3. Datenmodell (Entitäten, Beziehungen)
4. API-Design (Endpoints, Verben, Payloads)
5. Technologie-Empfehlung (mit Begründung)
6. Trade-offs (Was gewinnen wir? Was verlieren wir?)
7. Skalierungsstrategie (Wie wächst das System?)

Datenbank-Design

Designe ein Datenbankschema für [ANWENDUNG].

DATENBANK: [PostgreSQL/MySQL/MongoDB/...]

ENTITIES: [Liste der Entitäten]

FÜR JEDE ENTITY:

- Felder (Name, Typ, Constraints)
- Beziehungen (1:1, 1:N, N:M)
- Indizes (für erwartete Queries)

BERÜCKSICHTIGE:

- Normalisierung (3. Normalform oder bewusste Denormalisierung?)
- Soft Deletes (deleted_at statt physischem Löschen?)
- Audit-Trail (created_at, updated_at, created_by)
- Multi-Tenancy (wenn relevant)

ERSTELLE:

1. ER-Diagramm (als Text/Mermaid)
2. SQL CREATE TABLE Statements
3. Migrations-Dateien (für [Prisma/TypeORM/Alembic/...])
4. Seed-Daten (Testdaten)

Sprach-spezifische Tipps

Python

STIL: PEP 8, Type Hints (Python 3.12+), f-Strings

IMPORTS: Absolute Imports, keine Star-Imports

ASYNC: asyncio mit async/await wenn I/O-bound

DATENKLASSEN: Pydantic v2 für Validierung, dataclass für interne Strukturen

TESTING: pytest, parametrize für Varianten, fixtures für Setup

TypeScript

```
STIL: strict Mode, kein any, keine type assertions (as)
IMPORTS: Named Imports, Barrel-Files sparsam
ASYNC: Promises, async/await, keine Callbacks
VALIDIERUNG: Zod für Runtime-Validierung, TypeScript für
Compile-Time
TESTING: Vitest, describe/it, expect-Syntax
LINTING: ESLint + Prettier (konfiguriert)
```

Go

```
STIL: gofmt, Effective Go, keine generischen Interface{}-
Typen
ERROR HANDLING: Explicit error returns (if err != nil),
keine panic()
CONCURRENCY: Goroutines + Channels, context.Context für
Cancellation
TESTING: table-driven tests, testify für Assertions
STRUCTURE: cmd/ pkg/ internal/ nach Standard Go Layout
```

Rust

```
STIL: clippy, rustfmt, idiomatic Rust
ERROR HANDLING: Result<T, E>, thiserror für Custom Errors,
anyhow für Applications
OWNERSHIP: Borrowing statt Cloning wo möglich
ASYNC: tokio Runtime, async/await
TESTING: #[cfg(test)], unit tests im gleichen File,
integration tests in tests/
```

Anti-Patterns in der Code-Generierung

Anti-Pattern 1: Blind Copy-Paste

KI-generierter Code kann Bugs enthalten. Immer reviewen, nie blind einfügen.

Anti-Pattern 2: Zu große Prompts

“Schreib mir die komplette Anwendung” → funktioniert nicht. Bau sie Modul für Modul.

Anti-Pattern 3: Keine Iteration

Der erste Output ist selten perfekt. Iteriere:

```
"Der Code funktioniert, aber:  
1. Die Error-Messages sind zu generisch  
2. Das Logging fehlt  
3. Die Funktion ist zu lang (>50 Zeilen)  
Verbessere diese 3 Punkte."
```

Anti-Pattern 4: KI-Code nicht testen

KI-generierter Code muss genauso (oder strenger) getestet werden wie handgeschriebener Code. Vertraue nicht – verifiziere.

Anti-Pattern 5: Veraltete Patterns akzeptieren

KI-Modelle haben Wissens-Cutoffs. Code-Patterns können veraltet sein. Prüfe:

- Werden aktuelle API-Versionen verwendet?
- Sind deprecated Methoden im Code?
- Entspricht der Code aktuellen Best Practices?

Übungen

Übung 1: Der volle Code-Prompt

Wähle eine Aufgabe aus deinem Arbeitsalltag und schreibe einen vollständigen Code-Prompt (Aufgabe, Kontext, Tech-Stack, Architektur, Anforderungen, Qualität). Vergleiche das Ergebnis mit einem einfachen “Schreib mir X”-Prompt.

Übung 2: Code Review

Nimm ein eigenes Code-Stück (oder Open-Source-Code) und lass es reviewen. Stimmen die Findings? Gibt es False Positives?

Übung 3: Test-Generierung

Lass für eine bestehende Funktion Tests generieren. Laufen sie? Decken sie die wichtigen Edge Cases ab?

Übung 4: Bug-Analyse

Nimm einen echten Bug (z.B. aus einem GitHub Issue) und lass die KI eine Root Cause Analysis durchführen. Liegt sie richtig?

Kapitel 2: Agentic Coding Tools – Die neue Generation der Entwicklerwerkzeuge

2024 war das Jahr der Coding-Assistenten. 2025 war das Jahr der Coding-Agenten. 2026 ist das Jahr, in dem sie zum Standard geworden sind.

Der Unterschied: Ein Assistent schlägt Code vor, den du einfügst. Ein Agent schreibt Code, führt ihn aus, liest Fehlermeldungen, fixt Bugs und commitet – alles autonom. Du gibst die Richtung vor. Der Agent erledigt die Schrittarbeit.

Die Landschaft (Stand März 2026)

Tier 1: Agentic-First

Tool	Modell	Stärke	Preis
Claude Code (Anthropic)	Claude Opus 4 / Sonnet 4	Terminal-Agent, plant & handelt autonom, Git-integriert	API-basiert (pay per token)
Cursor	Multi-Model (Claude, GPT, Gemini)	IDE mit Agent-Mode, Composer, multi-file Edits	\$20/Monat (Pro)
Windsurf (Codeium)	Multi-Model	Cascade-Flow, kontextuelles Verständnis	\$15/Monat (Pro)
Devin (Cognition)	Proprietär	Vollautonomer Agent, eigene Sandbox	Enterprise

Tier 2: Assistenten mit Agent-Fähigkeiten

Tool	Modell	Stärke	Preis
GitHub Copilot	GPT-4o + Claude	IDE-integriert, Agent-Mode (Preview), Workspace-Kontext	\$10-\$39/Monat
Cline	Multi-Model (via API)	Open Source, VS-Code-Extension, volle Kontrolle	Kostenlos (+ API-Kosten)
Aider	Multi-Model (via API)	Terminal-basiert, Git-integriert, diff-basiert	Kostenlos (+ API-Kosten)
Continue	Multi-Model	Open Source, IDE-Extension, anpassbar	Kostenlos (+ API-Kosten)

Tier 3: Spezialisiert

Tool	Fokus	Stärke
Bolt.new	Full-Stack Web-	Generiert komplette Apps im Browser Apps
v0 (Vercel)	Frontend/UI	React-Komponenten aus Beschreibungen
Lovable	Full-Stack	Produkt-Building für Nicht-Entwickler
Replit Agent	Full-Stack	Deployment inklusive

Wie du in Coding-Agenten optimal promptest

Claude Code

Claude Code ist ein Terminal-basierter Agent, der direkt in deinem Repo arbeitet:

```
# Starten
claude

# In Claude Code prompten:
"Lies die README und erkläre mir die Architektur dieses Projekts."

"Implementiere eine REST-API für User-Management:
- CRUD-Endpoints (GET, POST, PUT, DELETE)
- Zod-Validierung
- Prisma ORM
- Schreibe Tests mit Vitest
- Committe am Ende mit aussagekräftiger Commit-Message."
```

Best Practices für Claude Code:

1. **CLAUDE.md nutzen** – Erstelle eine `CLAUDE.md`-Datei im Root deines Projekts mit Architektur, Konventionen und Befehlen. Claude Code liest sie automatisch.
2. **Kleine, klare Aufgaben** – “Implementiere Feature X” ist besser als “Baue die ganze App”.
3. **Lass ihn planen** – “Erstelle zuerst einen Plan, bevor du implementierst.”
4. **Tests zuerst** – “Schreibe zuerst die Tests, dann die Implementierung.”
5. **Git nutzen** – Claude Code commitet automatisch. Prüfe die Commits.

CLAUDE.md – Das Projektgedächtnis

Die wichtigste Datei für agentic Coding:


```

# Projektname

## Architektur
- Monorepo mit Turborepo
- Backend: Node.js + Express + TypeScript
- Frontend: Next.js 15 + React 19
- DB: PostgreSQL + Prisma

## Befehle
- `npm run build` - Alles bauen
- `npm run test` - Tests ausführen
- `npm run lint` - Linting
- `npm run dev` - Entwicklungsserver

## Konventionen
- Alle Dateien: kebab-case
- Alle Funktionen: camelCase
- Alle Typen: PascalCase
- Error Handling: Result-Pattern, kein throw
- Commits: Conventional Commits (feat:, fix:, chore:)
- Branch-Naming: feature/kurze-beschreibung

## Vermeide
- any-Types
- console.log in Production-Code
- Star-Imports
- Class Components (nur Functional Components)

```

Cursor

Cursor hat drei Modi: Tab-Completion, Chat und Composer (Agent).

Tab-Completion:

Cursor vervollständigt Code inline. Optimierte durch:

- Gute Variablenamen (die Completion wird besser)
- Kommentare als Hints: `// TODO: Validate email format`
- Typ-Annotationen (TypeScript/Python Type Hints)

Chat (Cmd+L):

```
Kontext: Markiere relevanten Code, dann frage:  
"Erkläre, warum dieser Code bei mehr als 1000 Items  
langsam wird und schlage eine Optimierung vor."
```

Composer / Agent Mode (Cmd+I):

```
"Erstelle eine neue API-Route /api/v2/users mit:  
- Pagination (cursor-based)  
- Filtering (nach Name, Email, Status)  
- Sorting (nach jedem Feld, ASC/DESC)  
- Rate Limiting (100 req/min)  
Nutze die bestehende Middleware aus src/middleware/.  
Schreibe Tests."
```

Cursor Rules (.cursorrules):

```
# .cursorrules  
You are a senior TypeScript developer.  
Always use strict TypeScript - no 'any'.  
Use Zod for runtime validation.  
Follow the repository's existing patterns.  
Write tests for every new function.  
Use descriptive variable names.  
Prefer composition over inheritance.
```

GitHub Copilot

Copilot hat sich von einem Autocomplete-Tool zu einem Agent entwickelt:

Copilot Chat (in VS Code):

```
@workspace Wie ist die Authentifizierung in diesem  
Projekt implementiert?  
  
@workspace Schreibe eine Migration, die eine  
"notifications"-Tabelle hinzufügt.
```

Copilot Agent Mode:

```
Implementiere folgendes GitHub Issue: #123  
Lies das Issue, verstehe die Anforderungen,  
implementiere, teste und erstelle einen PR.
```

Cline (Open Source)

Cline ist der Open-Source-Champion: Volle Kontrolle, jedes Modell, keine Vendor-Lock-in.

```
# In Cline (VS Code Extension):  
"Analysiere die Performance dieses Projekts:  
1. Lies alle API-Routes  
2. Identifiziere N+1 Query-Probleme  
3. Schlage Optimierungen vor  
4. Implementiere die wichtigste Optimierung  
5. Schreibe einen Benchmark-Test"
```

Cline Best Practices:

- `.clinerules` für Projekt-Konventionen
- Auto-approve für sichere Operationen (read, list)
- Manual-approve für Schreiboperationen
- API-Budget setzen (Token-Limit pro Task)

Aider

Terminal-basiert, Git-integriert, minimal:

```
# Starten mit einem Modell
aider --model claude-sonnet-4-20250514

# In Aider:
/add src/api/users.ts src/models/user.ts
> Füge Pagination zur Users-API hinzu.
> Nutze cursor-based Pagination mit Prisma.
```

Aider-Stärken:

- Versteht Git nativ (zeigt Diffs, commitet)
- Funktioniert mit jedem Modell via API
- Lightweight, keine IDE nötig
- Architect-Modus: Erst planen (mit starkem Modell), dann implementieren (mit schnellem Modell)

Welches Tool für welchen Zweck?

Situation	Empfehlung
Kleine Änderungen, quick Fixes	Cursor Tab-Completion, Copilot
Neue Features (multi-file)	Cursor Composer, Claude Code
Große Refactorings	Claude Code, Aider
Code verstehen (onboarding)	Claude Code, Cursor Chat
Prototyping (0 → 1)	Bolt.new, v0, Cursor Composer
Open Source / Budget	Cline, Aider, Continue
Enterprise / Compliance	GitHub Copilot Enterprise, Cursor Business

Effektive Strategien für alle Tools

1. Kontext ist King

Alle Tools profitieren von Kontext. Je mehr relevante Dateien du referenzierst oder in den Kontext gibst, desto besser das Ergebnis.

2. Inkrementell arbeiten

Nicht “baue mir die ganze App” – sondern Schritt für Schritt. Ein Feature nach dem anderen. Ein Modul nach dem anderen.

3. Tests als Qualitätsgate

Lass den Agent Tests schreiben. Dann: Tests ausführen. Wenn sie rot sind: Agent fixen lassen. Die Feedback-Loop ist der Schlüssel.

4. Reviewe alles

Auch agentic-generierter Code muss reviewed werden. Git Diff lesen. Verstehen, was geändert wurde. Nicht blind mergen.

5. Versionskontrolle als Safety Net

Git ist dein Undo-Button. Committe oft. Branch für experimentelle Änderungen. Wenn der Agent Mist baut: `git checkout .`

Übungen

Übung 1: CLAUDE.md erstellen

Erstelle eine CLAUDE.md für ein eigenes Projekt (oder ein Open-Source-Projekt, das du kennst). Welche Informationen braucht ein KI-Agent, um guten Code zu schreiben?

Übung 2: Tool-Vergleich

Nimm dieselbe Aufgabe und löse sie mit zwei verschiedenen Tools (z.B. Cursor + Aider oder Copilot + Claude Code). Vergleiche die Ergebnisse und den Workflow.

Übung 3: Agent-Modus testen

Nutze den Agent-Modus eines Tools (Cursor Composer, Claude Code, Copilot Agent) für ein echtes Feature. Wie viel musstest du manuell nacharbeiten?

Übung 4: Fehlerbehebung

Gib einem Coding-Agent einen Bug-Report und lass ihn autonom debuggen. Wie weit kommt er ohne dein Eingreifen?

Kapitel 3: Die LLM-APIs – Dein Zugang zur KI-Power

Bisher hast du KI über Chat-Interfaces genutzt. Jetzt nutzt du sie programmatisch – über APIs. Das bedeutet: Du schickst HTTP-Requests und bekommst strukturierte Responses. Automatisiert. Skalierbar. Integriert in deine Software.

Die großen Drei (und die Challenger)

Anthropic (Claude)

Modell	Kontext	Stärke	Input / Output (pro 1M Token)
Claude Opus 4.6	200K (1M beta)	#1 Reasoning, 80.8% SWE-bench, Code	\$5 / \$25
Claude Sonnet 4.5	200K	Bestes Preis-Leistung, 77-82% SWE-bench	\$3 / \$15
Claude Haiku 4.5	200K	Schnell, günstig, agentic Loops	\$0.25 / \$1.25

Besondere Features:

- **Extended Thinking** – Das Modell “denkt nach” bevor es antwortet. Steuerbar über `effort`-Parameter. Thinking-Tokens als Output abgerechnet.

- **Tool Use** – Natives Function Calling mit JSON-Schema. Plus: Web Search, Code Execution, Computer Use, programmatic Tool Calling.
- **Computer Use** – Claude kann einen Computer steuern (Screenshots, Klicks, Tippen).
- **Prompt Caching** – Wiederholt genutzte Prompt-Teile werden gecacht (90% günstiger).
- **Batches API** – Massenhaft Requests mit 50% Rabatt (24h SLA).
- **Vision** – Bilder, PDFs, Charts, Diagramme als Input.
- **Citationen** – Claude kann Quellenangaben in Dokumenten referenzieren.
- **Compaction API** (beta) – Server-seitige Kontext-Zusammenfassung für endlose Konversationen.
- **1M Token Context Window** (beta für Opus 4.6).

OpenAI (GPT)

Modell	Kon-text	Stärke	Input / Output (pro 1M Token)
GPT-5.2 (xhigh)	1M	#1 Coding-Benchmarks (89% LiveCodeBench)	\$1.75 / \$14
GPT-5 mini	200K	Schnell, günstig, starkes Allround	\$0.25 / \$2
GPT-5 nano	128K	Ultra-günstig, Edge-De- ployment	\$0.05 / \$0.40
GPT-4o	128K	Multimodal (Text + Bild + Audio)	\$2.50 / \$10
o3	200K	Deep Reasoning, Mathe- matik	\$10 / \$40

Besondere Features:

- **Responses API** – Ersetzt Chat Completions für neue Projekte. Agentic by default, Multi-Tool-Loop in einer Anfrage.

- **Structured Outputs** – Garantiert valides JSON mit `strict: true`.
- **Function Calling** – Nativ, parallel (mehrere Tools gleichzeitig).
- **Agents SDK** – Open-Source Multi-Agent-Orchestrierung mit MCP-Support.
- **Realtime API** – Audio-Input und -Output in Echtzeit.
- **Fine-tuning** – Für GPT-5 mini und GPT-4o verfügbar.

Google (Gemini)

Modell	Kon- text	Stärke	Input / Output (pro 1M Token)
Gemini 3.1 Pro	1M	Reasoning, Cross- Language	\$2 / \$12
Gemini 3 Flash	1M	Schnell, günstig	\$0.50 / \$3
Gemini 2.5 Pro	1M	Bewährt, riesiger Kontext	\$1.25 / \$10
Gemini 2.0 Flash-Lite	1M	Ultra-günstig	\$0.075 / \$0.30

Besondere Features:

- **1M Token Kontext** – Gesamte Codebases in einem Request.
- **Großzügigstes Free Tier** – Bis zu 1.000 Anfragen/Tag kostenlos.
- **Grounding mit Google Search** – Modell kann googeln.
- **Code Execution** – Führt Python-Code in Sandbox aus.
- **Thinking Budget** – Steuere, wie viel das Modell “nachdenken” soll.
- **MCP-Support** – Nativ integriert.

Open Source / Alternative

Modell	Parameter	Stärke	Zugang
Llama 4 (Meta)	Scout: 17B aktiv (109B gesamt), Maverick: 17B aktiv (400B gesamt)	Multimodal, Open Source	Lokal / Together / Groq
Deep-Seek-V3.2	685B (37B aktiv)	Coding, Reasoning, \$0.28/1M In	API / Lokal
Deep-Seek-R1	685B	Reasoning (o1-Level)	API / Lokal
Qwen 3	Diverse	Multimodal, mehrsprachig	API / Lokal
Mistral Large 2	123B	Europa, mehrsprachig, Code	API / Lokal
Codestral	22B	Speziell für Code	API / Lokal

Dein erster API-Call

Python (Anthropic SDK)

```
import anthropic

client = anthropic.Anthropic() # ANTHROPIC_API_KEY aus Env

message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[
        {
            "role": "user",
            "content": "Erkläre mir in 3 Sätzen, was eine
API ist."
        }
    ]
)

print(message.content[0].text)
```

Python (OpenAI SDK)

```
from openai import OpenAI

client = OpenAI() # OPENAI_API_KEY aus Env

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": "Erkläre mir in 3 Sät-
zen, was eine API ist."}
    ]
)

print(response.choices[0].message.content)
```

TypeScript (Anthropic SDK)

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

const message = await client.messages.create({
  model: "claude-sonnet-4-6",
  max_tokens: 1024,
  messages: [
    {
      role: "user",
      content: "Erkläre mir in 3 Sätzen, was eine API ist."
    }
  ]
});

console.log(message.content[0].text);
```

System-Prompts

System-Prompts definieren das Verhalten des Modells über die gesamte Konversation:

```
message = client.messages.create(
  model="claude-sonnet-4-6",
  max_tokens=2048,
  system="""Du bist ein Senior Python Developer mit 15
Jahren Erfahrung.
Du schreibst sauberen, typisierten Python-Code nach PEP 8.
Du bevorzugst funktionale Patterns über OOP.
Du erklärst deine Designentscheidungen kurz.
Du schreibst immer Tests.""",
  messages=[
    {"role": "user", "content": "Implementiere eine Re-
try-Logik mit exponential backoff."}
  ]
)
```

Streaming

Für UX-freundliche Responses – Text erscheint Wort für Wort:

```
# Anthropic Streaming
with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Schreibe eine
Kurzgeschichte."}]
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

```
# OpenAI Streaming
stream = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Schreibe eine
Kurzgeschichte."}],
    stream=True
)
for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="",
flush=True)
```

Structured Output

Das Modell gibt garantiert valides JSON zurück:

Anthropic (Tool Use für strukturiertes JSON)

```

import anthropic

client = anthropic.Anthropic()

message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    tools=[{
        "name": "analyze_sentiment",
        "description": "Analysiere das Sentiment eines Textes",
        "input_schema": {
            "type": "object",
            "properties": {
                "sentiment": {
                    "type": "string",
                    "enum": ["positiv", "neutral", "negativ"]
                },
                "confidence": {
                    "type": "number",
                    "minimum": 0,
                    "maximum": 1
                },
                "keywords": {
                    "type": "array",
                    "items": {"type": "string"}
                }
            },
            "required": ["sentiment", "confidence", "keywords"]
        }
    }],
    tool_choice={"type": "tool", "name": "analyze_sentiment"},
    messages=[{
        "role": "user",
        "content": "Analysiere: 'Das neue Update ist fantastisch, aber der Preis ist zu hoch.'"
    }]
)

# Ergebnis ist garantiert valides JSON nach Schema
result = message.content[0].input
# {"sentiment": "neutral", "confidence": 0.7, "keywords": ["Update", "fantastisch", "Preis", "hoch"]}

```

Prompt Caching (Anthropic)

Spart bis zu 90% bei wiederholten Prompts:

```
message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system=[
        {
            "type": "text",
            "text": "Du bist ein Experte für deutsches
Steuerrecht...",
            "cache_control": {"type": "ephemeral"} # Die-
sen Teil cachen
        }
    ],
    messages=[{"role": "user", "content":
"Wie funktioniert die Umsatzsteuervoranmeldung?"}]
)

# Beim zweiten Call mit demselben System-Prompt:
# Cache Hit → 90% günstiger für den gecachten Teil
```


Vision (Bilder als Input)

```
import base64

# Bild als Base64
with open("screenshot.png", "rb") as f:
    image_data =
base64.standard_b64encode(f.read()).decode("utf-8")

message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{
        "role": "user",
        "content": [
            {
                "type": "image",
                "source": {
                    "type": "base64",
                    "media_type": "image/png",
                    "data": image_data
                }
            },
            {
                "type": "text",
                "text": "Erstelle den HTML/CSS-Code, der
dieses Design nachbaut."
            }
        ]
    }]
)
```

Kosten optimieren

Strategie 1: Das richtige Modell wählen

Einfache Aufgaben (Klassifikation, Extraktion) → Haiku / GPT-4o-mini
Mittlere Aufgaben (Texte, Code) → Sonnet / GPT-4o
Komplexe Aufgaben (Reasoning, Architektur) → Opus / o3

Strategie 2: Prompt Caching

Wiederholt genutzte System-Prompts und Kontexte cachen. Spart 90% auf gecachte Tokens.

Strategie 3: Batches

Nicht zeitkritische Anfragen sammeln und als Batch senden. 50% günstiger.

Strategie 4: Token-Budget

`max_tokens` auf das Minimum setzen. Nicht 4096, wenn 500 reichen.

Strategie 5: Prompt-Optimierung

Kürzere Prompts = weniger Input-Tokens = günstiger. Unnötige Wiederholungen entfernen.

Übungen

Übung 1: Erster API-Call

Erstelle einen API-Key bei Anthropic oder OpenAI. Schreibe deinen ersten API-Call in Python oder TypeScript. Lass dir einen Witz erzählen.

Übung 2: System-Prompt

Schreibe einen System-Prompt für einen spezialisierten Assistenten (z.B. Code-Reviewer, Übersetzer, Datenanalyst). Teste ihn mit verschiedenen User-Messages.

Übung 3: Structured Output

Nutze Tool Use oder Structured Outputs, um das Sentiment von 10 Produktbewertungen zu analysieren. Gib das Ergebnis als JSON zurück.

Übung 4: Kosten berechnen

Schätze die API-Kosten für ein Szenario: 1.000 Kundenanfragen pro Tag, je ~500 Input-Tokens und ~200 Output-Tokens. Vergleiche die Kosten bei Claude Sonnet, GPT-4o und Gemini Flash.

Kapitel 4: Programmatisches Prompting – Prompts als Code

In Kapitel 3 hast du API-Calls gemacht. Einzelne Requests, einzelne Antworten. Jetzt bauen wir Systeme: Prompt-Templates, Variablen, Pipelines, Fehlerbehandlung und Multi-Turn-Konversationen.

Prompt-Templates

Hartcodierte Prompts sind wie hartcodierte Strings – sie funktionieren, aber sie skalieren nicht.

Python: Prompt-Templates mit f-Strings

```
def create_analysis_prompt(
    text: str,
    language: str = "Deutsch",
    detail_level: str = "mittel"
) -> str:
    return f"""Analysiere folgenden Text auf {language}.

TEXT:
\"\"\"
{text}
\"\"\"

DETAILGRAD: {detail_level}
- "kurz": 3 Bullet Points
- "mittel": 1 Absatz Zusammenfassung + 5 Key Findings
- "ausführlich": Vollständige Analyse mit Zitaten

Antworte auf {language}."""
```

Fortgeschritten: Template-Klasse

```
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class PromptTemplate:
    system: str
    user_template: str
    model: str = "claude-sonnet-4-6"
    max_tokens: int = 1024
    temperature: float = 0.0

    def render(self, **kwargs) -> str:
        return self.user_template.format(**kwargs)

# Templates definieren
SENTIMENT_TEMPLATE = PromptTemplate(

    system="Du bist ein Sentiment-Analyse-Experte. Antworte nur mit JSON.",
    user_template="""Analysiere das Sentiment folgender {count} Texte:

    {texts}

    Für jeden Text: sentiment (positiv/neutral/negativ), score (-1 bis 1), begründung (1 Satz).""",
    temperature=0.0
)

# Nutzen
prompt = SENTIMENT_TEMPLATE.render(
    count=3,
    texts="1. Super Produkt!\n2. Geht so.\n3. Nie wieder."
)
```

Multi-Turn-Konversationen

LLM-APIs sind zustandslos. Für Konversationen musst du die History selbst verwalten:

```

class Conversation:
    def __init__(self, client, system: str, model: str = "claude-sonnet-4-6"):
        self.client = client
        self.system = system
        self.model = model
        self.messages: list[dict] = []

    def send(self, user_message: str) -> str:
        self.messages.append({"role": "user", "content": user_message})

        response = self.client.messages.create(
            model=self.model,
            max_tokens=2048,
            system=self.system,
            messages=self.messages
        )

        assistant_message = response.content[0].text
        self.messages.append({"role": "assistant", "content": assistant_message})

        return assistant_message

    def reset(self):
        self.messages = []

# Nutzung
conv = Conversation(
    client=anthropic.Anthropic(),
    system="Du bist ein Python-Tutor. Erkläre Konzepte mit Beispielen."
)

answer1 = conv.send("Was ist eine List Comprehension?")
answer2 = conv.send("Zeig mir ein komplexeres Beispiel mit Bedingung.")
# Das Modell kennt die vorherige Frage und baut darauf auf

```

Fehlerbehandlung und Retry-Logik

APIs können fehlschlagen. Rate Limits, Timeouts, Server-Fehler. Robuster Code fängt das ab:

```
import time
import anthropic

def call_with_retry(
    client: anthropic.Anthropic,
    max_retries: int = 3,
    **kwargs
) -> anthropic.types.Message:
    for attempt in range(max_retries):
        try:
            return client.messages.create(**kwargs)

        except anthropic.RateLimitError:
            wait = 2 ** attempt # 1s, 2s, 4s
            print(f"Rate limit. Warte {wait}s...")
            time.sleep(wait)

        except anthropic.APITimeoutError:
            wait = 2 ** attempt
            print(f"Timeout. Warte {wait}s...")
            time.sleep(wait)

        except anthropic.APIStatusError as e:
            if e.status_code >= 500: # Server-Fehler
                wait = 2 ** attempt
                print(f"Server error {e.status_code}. Warte {wait}s...")
                time.sleep(wait)
            else:
                raise # Client-Fehler nicht retieren (400, 401, etc.)

    raise Exception(f"Fehlgeschlagen nach {max_retries} Versuchen")
```


Batch-Verarbeitung

Wenn du viele Items verarbeiten musst (z.B. 1.000 Produktbeschreibungen analysieren):

```
import asyncio
import anthropic

async def process_batch(
    items: list[str],
    prompt_template: str,
    concurrency: int = 5
) -> list[str]:
    client = anthropic.AsyncAnthropic()
    semaphore = asyncio.Semaphore(concurrency)
    results = []

    async def process_one(item: str) -> str:
        async with semaphore:
            message = await client.messages.create(
                model="claude-haiku-4-5-20251001",
                max_tokens=256,
                messages=[{
                    "role": "user",
                    "content":
prompt_template.format(item=item)
                }]
            )
            return message.content[0].text

    tasks = [process_one(item) for item in items]
    results = await asyncio.gather(*tasks)
    return results

# Nutzung
items = ["Produkt A ist super", "Produkt B ist okay", ...]
results = asyncio.run(process_batch(
    items,
    "Klassifiziere das Sentiment: '{item}'. Antwort: posi-
tiv/neutral/negativ",
    concurrency=10
))
```

Antwort-Parsing

JSON-Parsing

```
import json

def parse_json_response(text: str) -> dict:
    """Extrahiere JSON aus einer LLM-Antwort."""
    # Versuche direktes Parsing
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass

    # Suche nach JSON-Block in Markdown
    import re
    json_match = re.search(r'```(?:json)?\s*\n(?:.*?)\n```',
text, re.DOTALL)
    if json_match:
        return json.loads(json_match.group(1))

    # Suche nach JSON-Objekt im Text
    json_match = re.search(r'\{.*\}', text, re.DOTALL)
    if json_match:
        return json.loads(json_match.group(0))

    raise ValueError(f"Kein JSON gefunden in: {text[:200]}.
..")
```

Besser: Structured Output nutzen (siehe Kapitel 3)

Statt JSON aus Freitext zu parsen, nutze Tool Use oder Structured Outputs. Das ist zuverlässiger.

Prompt-Chaining programmatisch

Mehrere LLM-Calls hintereinander, wobei der Output des einen der Input des nächsten ist:

```

async def research_and_write(topic: str) -> str:
    client = anthropic.AsyncAnthropic()

    # Schritt 1: Recherche
    research = await client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=2048,
        messages=[{
            "role": "user",
            "content": f"Recherchiere die 5 wichtigsten
Aspekte von: {topic}"
        }]
    )
    research_text = research.content[0].text

    # Schritt 2: Outline erstellen
    outline = await client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=1024,
        messages=[{
            "role": "user",
            "content": f""Basierend auf dieser Recherche:
{research_text}

Erstelle ein Outline für einen Artikel (5 Abschnitte).""
        }]
    )
    outline_text = outline.content[0].text

    # Schritt 3: Artikel schreiben
    article = await client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=4096,
        messages=[{
            "role": "user",
            "content": f""Schreibe einen Artikel nach
folgendem Outline:
{outline_text}

Stil: Sachlich, informativ, 800 Wörter.""
        }]
    )

    return article.content[0].text

```

Evaluierung und Monitoring

Prompt-Qualität messen

```

from dataclasses import dataclass

@dataclass
class EvalResult:
    input: str
    expected: str
    actual: str
    correct: bool
    latency_ms: float
    tokens_used: int

def evaluate_prompt(
    client,
    test_cases: list[dict], # [{"input": ..., "expected": ...}]
    prompt_template: str,
    model: str
) -> list[EvalResult]:
    results = []

    for case in test_cases:
        start = time.time()
        message = client.messages.create(
            model=model,
            max_tokens=256,
            messages=[{
                "role": "user",
                "content": prompt_template.format(**case)
            }]
        )
        latency = (time.time() - start) * 1000
        actual = message.content[0].text.strip()

        results.append(EvalResult(
            input=case["input"],
            expected=case["expected"],
            actual=actual,
            correct=actual.lower() == case["expected"].lower(),
            latency_ms=latency,
            tokens_used=message.usage.input_tokens + message.usage.output_tokens
        ))

    accuracy = sum(1 for r in results if r.correct) / len(results)

```

```
avg_latency = sum(r.latency_ms for r in results) /  
len(results)  
print(f"Accuracy: {accuracy:.1%} | Avg Latency: {avg_la  
tency:.0f}ms")  
  
return results
```

Best Practices

1. Prompt und Code trennen

Prompts in eigene Dateien (`.txt` , `.md` , YAML) – nicht in den Code inline.

2. Versionierung

Prompts versionieren (in Git oder einer Prompt-Registry). A/B-Testing ermöglichen.

3. Logging

Jeden API-Call loggen: Input, Output, Tokens, Latenz, Modell, Timestamp. Unverzichtbar für Debugging und Kostenoptimierung.

4. Testfälle pflegen

Mindestens 20 Testfälle pro Prompt-Template. Bei Änderungen: Regression testen.

5. Graceful Degradation

Wenn die API ausfällt: Fallback auf gecachte Antworten oder ein günstigeres Modell.

Übungen

Übung 1: Template-Bibliothek

Erstelle eine PromptTemplate-Klasse und 3 Templates für verschiedene Aufgaben (Sentiment, Zusammenfassung, Übersetzung).

Übung 2: Konversations-Manager

Implementiere eine Conversation-Klasse mit History, Token-Counting und Max-History-Länge.

Übung 3: Batch-Verarbeitung

Verarbeite 50 Texte parallel mit asyncio. Miss die Gesamtdauer vs. sequentielle Verarbeitung.

Übung 4: Evaluierung

Erstelle 10 Testfälle für ein Sentiment-Analyse-Template. Wie hoch ist die Accuracy?

Kapitel 5: RAG – Retrieval Augmented Generation

Das wichtigste Pattern für professionelle KI-Anwendungen. Punkt.

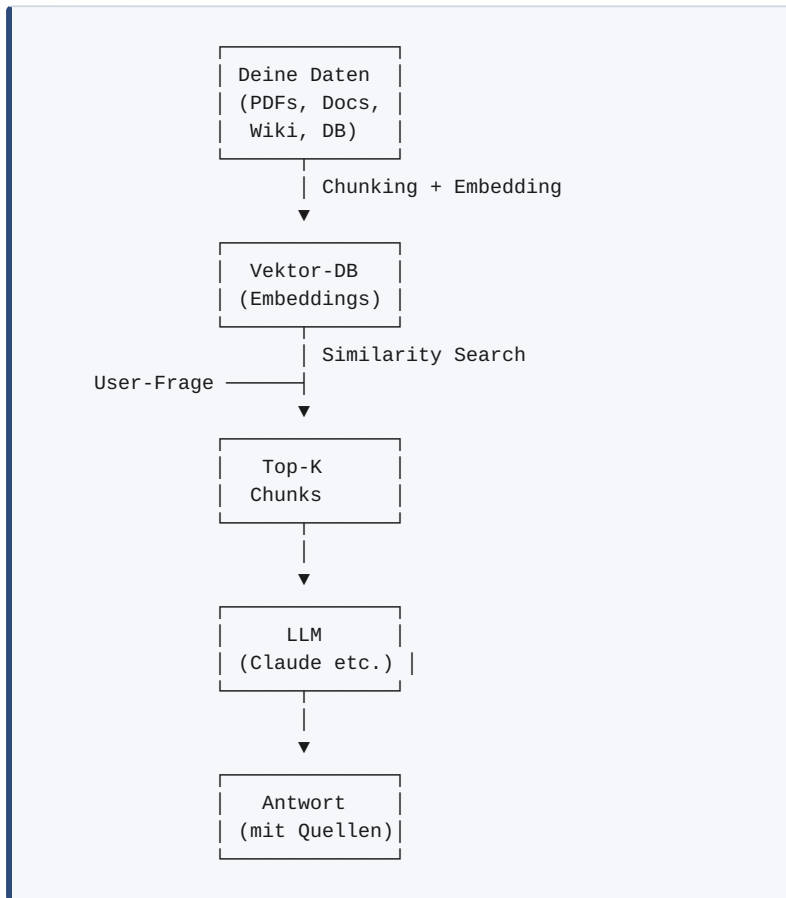
RAG löst das fundamentale Problem von LLMs: Sie wissen nur, was in ihren Trainingsdaten steht. Dein Unternehmens-Wiki? Deine Produktdokumentation? Deine Verträge? Davon weiß das Modell nichts.

RAG ändert das. Du reichst dem Modell die relevanten Informationen zusammen mit der Frage – und es antwortet basierend auf DEINEN Daten.

Wie RAG funktioniert

1. INDEXIERUNG (einmalig):
Dokumente → Chunks → Embeddings → Vektor-DB
2. ABFRAGE (bei jeder Frage):
User-Frage → Embedding → Ähnlichkeitssuche in Vektor-DB
→ Top-K relevante Chunks abrufen
3. GENERATION (bei jeder Frage):
System-Prompt + relevante Chunks + User-Frage
→ LLM generiert Antwort basierend auf den Chunks

Visuell:



Schritt 1: Dokumente chunken

Chunking ist die wichtigste Entscheidung im RAG-Stack. Zu große Chunks = zu viel Noise. Zu kleine Chunks = fehlender Kontext.

Chunking-Strategien

Strategie	Beschreibung	Wann nutzen
Fixed Size	Alle 500 Tokens schneiden	Einfach, aber naiv
Sentence-based	An Satzgrenzen schneiden	Bessere semantische Einheiten
Paragraph-based	An Absatzgrenzen schneiden	Gut für strukturierte Dokumente
Recursive	Versuche große Teiler (§, \n\n, \n, .)	Standard-Empfehlung, flexibel
Semantic	Embedding-basiert, split bei Themenwechsel	Beste Qualität, teuerste Berechnung
Document-aware	Markdown-Header, HTML-Tags	Ideal für strukturierte Formate

Python-Beispiel: Recursive Chunking

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,          # Zeichen pro Chunk
    chunk_overlap=200,        # Überlappung (Kontext-Brücke)
    separators=["\n\n", "\n", ". ", " ", ""],
    length_function=len
)

chunks = splitter.split_text(document_text)
# Ergebnis: Liste von Text-Chunks mit Überlappung
```

Chunk-Optimierung

```
# Metadata zu jedem Chunk hinzufügen
chunked_docs = []
for i, chunk in enumerate(chunks):
    chunked_docs.append({
        "text": chunk,
        "metadata": {
            "source": "handbuch_v2.pdf",
            "page": calculate_page(i),
            "chunk_index": i,
            "total_chunks": len(chunks),
            "section": extract_section_header(chunk)
        }
    })
```

Schritt 2: Embeddings erstellen

Embeddings wandeln Text in Vektoren um – mathematische Repräsentationen der Bedeutung.

Embedding-Modelle (Stand 2026)

Modell	Anbieter	Dimensionen	Stärke		Kosten
text-embedding-3-large	OpenAI	3072	Bestes	All-round	\$0.13 / 1M Token
text-embedding-3-small	OpenAI	1536	Günstig, genug	gut	\$0.02 / 1M Token
voyage-3-large	Voyage AI	1024	Sehr gut	für Code + Text	\$0.18 / 1M Token
embed-v4.0	Cohere	1024	Multilingual	stark	\$0.10 / 1M Token
nomic-embed-text	Nomic	768	Open Source, lokal nutzbar		Kostenlos (lokal)
mxbai-embed-large	Mixed-bread	1024	Deutsch besonders stark		Kostenlos (lokal)

Python: Embeddings erstellen (OpenAI)

```
from openai import OpenAI

client = OpenAI()

def get_embeddings(texts: list[str]) -> list[list[float]]:
    response = client.embeddings.create(
        model="text-embedding-3-small",
        input=texts
    )
    return [item.embedding for item in response.data]

# Alle Chunks embedden
chunk_texts = [chunk["text"] for chunk in chunked_docs]
embeddings = get_embeddings(chunk_texts)
```

Schritt 3: Vektor-Datenbank

Vektor-Datenbanken speichern Embeddings und ermöglichen schnelle Ähnlichkeitssuche.

Vektor-DB-Vergleich

DB	Typ	Stärke	Preis
Pinecone	Cloud (Managed)	Einfachste Setup, Serverless	Free Tier + Pay-per-use
Weaviate	Cloud + Self-hosted	Hybrid-Suche (Vektor + Keyword)	Free Tier + Managed
Qdrant	Cloud + Self-hosted	Performance, Rust-basiert	Free Tier + Self-hosted
ChromaDB	Lokal / Embedded	Einfachst für Prototypen	Kostenlos
pgvector	PostgreSQL Extension	In bestehende DB integriert	Kostenlos
SQLite-vec	SQLite Extension	Minimal, embedded	Kostenlos

Python: ChromaDB (einfachster Start)

```
import chromadb

# Client erstellen (lokal, persistent)
client = chromadb.PersistentClient(path="./chroma_db")

# Collection erstellen
collection = client.get_or_create_collection(
    name="meine_docs",
    metadata={"hnsw:space": "cosine"}
)

# Dokumente hinzufügen
collection.add(
    documents=[chunk["text"] for chunk in chunked_docs],
    metadatas=[chunk["metadata"] for chunk in chunked_docs]
,
    ids=[f"chunk_{i}" for i in range(len(chunked_docs))]
)

# Suchen
results = collection.query(
    query_texts=["Wie funktioniert die Rückgabe?"],
    n_results=5
)
```

Python: pgvector (in bestehender PostgreSQL)

```
# SQL: Extension aktivieren
# CREATE EXTENSION vector;
# CREATE TABLE documents (
#     id SERIAL PRIMARY KEY,
#     content TEXT,
#     embedding vector(1536),
#     metadata JSONB
# );
# CREATE INDEX ON documents USING ivfflat (embedding vector_cosine_ops);

import psycopg2

def search_similar(query_embedding, limit=5):
    conn = psycopg2.connect("postgresql://...")
    cur = conn.cursor()
    cur.execute("""
        SELECT content, metadata, 1 - (embedding <=> %s::vector)
        AS similarity
        FROM documents
        ORDER BY embedding <=> %s::vector
        LIMIT %s
    """, (query_embedding, query_embedding, limit))
    return cur.fetchall()
```


Schritt 4: RAG-Pipeline zusammenbauen

```

import anthropic
from openai import OpenAI

openai_client = OpenAI()
anthropic_client = anthropic.Anthropic()

def rag_query(question: str, collection, top_k: int = 5) -
> str:
    # 1. Frage embedden
    q_embedding = openai_client.embeddings.create(
        model="text-embedding-3-small",
        input=[question]
    ).data[0].embedding

    # 2. Ähnliche Chunks finden
    results = collection.query(
        query_embeddings=[q_embedding],
        n_results=top_k
    )

    # 3. Kontext zusammenbauen
    context = "\n\n---\n\n".join(results["documents"][0])
    sources = [m["source"] for m in results["metadatas"][0]
]

    # 4. LLM mit Kontext befragen
    message = anthropic_client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=2048,
        system="""Beantworte Fragen basierend auf dem be-
reitgestellten Kontext.
Wenn die Antwort nicht im Kontext steht, sag das ehrlich.
Zitiere relevante Stellen aus dem Kontext.""",
        messages=[{
            "role": "user",
            "content": f"""KONTEXT:
{context}

FRAGE: {question}

Beantworte die Frage basierend auf dem Kontext. Nenne die
Quellen."""
        }]
    )

    return message.content[0].text

```

Fortgeschrittene RAG-Techniken

Hybrid Search (Vektor + Keyword)

Kombiniere semantische Suche mit klassischer Keyword-Suche:

```
# Weaviate Hybrid Search
results = weaviate_client.query.get(
    "Document", ["content", "source"]
).with_hybrid(
    query="Rückgaberecht bei Online-Kauf",
    alpha=0.7 # 0 = nur Keyword, 1 = nur Vektor
).with_limit(5).do()
```

Reranking

Nach der Suche: Sortiere die Ergebnisse mit einem Reranker-Modell:

```
from cohere import Client

co = Client()

# Erste Suche: Vektor-DB (schnell, breit)
initial_results = collection.query(query_texts=[question],
n_results=20)

# Reranking: Präzisere Sortierung
reranked = co.rerank(
    model="rerank-v3.5",
    query=question,
    documents=initial_results["documents"][0],
    top_n=5
)

# Ergebnis: Die 5 relevantesten Chunks, besser sortiert
```

Query Expansion

Die User-Frage umformulieren für bessere Treffer:

```
# LLM generiert alternative Suchanfragen
expansion = anthropic_client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=256,
    messages=[{
        "role": "user",
        "content": f"""Generiere 3 alternative Formulierung
        en für diese Suchanfrage:
        "{question}"
        Nur die Alternativen, eine pro Zeile."""
    }]
)

# Suche mit allen Varianten, dedupliziere Ergebnisse
queries = [question] + expansion.content[0].text.strip().split("\n")
```

Contextual Retrieval (Anthropic)

Vor dem Embedden: Kontext zum Chunk hinzufügen:

```
# Für jeden Chunk: LLM generiert einen Kontextsatz
context_prompt = f"""Hier ist das gesamte Dokument:
{full_document}

Hier ist ein Chunk daraus:
{chunk}

Schreibe einen kurzen Satz (max. 20 Wörter), der erklärt,
wo sich dieser Chunk im Gesamtdokument einordnet."""

# Der Kontextsatz wird dem Chunk vorangestellt:
# "Dieser Abschnitt beschreibt die Rückgabebedingungen für
# Online-Käufe."
# + Original-Chunk-Text
# → Bessere Embeddings, bessere Suchergebnisse
```

RAG Anti-Patterns

Anti-Pattern 1: Zu große Chunks

→ Das Modell bekommt irrelevante Informationen und wird verwirrt.

Anti-Pattern 2: Kein Overlap

→ Informationen, die über Chunk-Grenzen gehen, werden nicht gefunden.

Anti-Pattern 3: Blindes Vertrauen

→ Das Modell kann Informationen aus den Chunks falsch interpretieren. Immer Quellen angeben und verifizierbar machen.

Anti-Pattern 4: Alles in den Kontext

→ 20 Chunks à 1000 Tokens = 20k Tokens Kontext. Das Modell findet die Nadel im Heuhaufen nicht. Top 3-5 reichen meist.

Übungen

Übung 1: Einfaches RAG

Nimm 3-5 PDF-Dokumente, chunke sie, erstelle Embeddings und baue eine RAG-Pipeline mit ChromaDB. Stelle Fragen an deine Dokumente.

Übung 2: Chunking-Vergleich

Chunke dasselbe Dokument mit verschiedenen Strategien (500 vs. 1000 Zeichen, mit/ohne Overlap). Vergleiche die Suchergebnisse.

Übung 3: Hybrid Search

Implementiere eine Hybrid-Suche, die Vektor-Suche mit Keyword-Suche kombiniert. Bei welchen Fragen ist Hybrid besser?

Übung 4: Evaluierung

Erstelle 10 Frage-Antwort-Paare für deine Dokumente. Wie oft gibt das RAG-System die richtige Antwort? Wo scheitert es?

Kapitel 6: Tool Use und Function Calling – KI greift in die echte Welt

Bisher generierte KI Text. Jetzt greift sie in die Welt ein: Datenbanken abfragen, APIs aufrufen, E-Mails senden, Dateien erstellen. Das ist Tool Use – und es verwandelt ein Sprachmodell in einen handlungsfähigen Agenten.

Was ist Tool Use?

Du definierst Funktionen (Tools) mit Schema. Das Modell entscheidet, wann welches Tool aufgerufen wird, und generiert die Parameter. Du führst die Funktion aus und gibst das Ergebnis zurück. Das Modell verarbeitet das Ergebnis und antwortet dem User.

```
User: "Wie ist das Wetter in Berlin?"  
↓  
LLM entscheidet: Tool "get_weather" mit {"city": "Berlin"}  
↓  
Dein Code führt get_weather("Berlin") aus → {"temp": 12,  
"condition": "bewölkt"}  
↓  
LLM: "In Berlin sind es 12°C und bewölkt."
```

Tool-Definition (Anthropic)


```

import anthropic

client = anthropic.Anthropic()

tools = [
    {
        "name": "get_weather",
        "description": "Ruft die aktuelle Wettervorhersage
für eine Stadt ab. "
        "Nutze dieses Tool wenn der User
nach Wetter fragt.",
        "input_schema": {
            "type": "object",
            "properties": {
                "city": {
                    "type": "string",
                    "description": "Name der Stadt, z.B.
'Berlin' oder 'München'"
                },
                "unit": {
                    "type": "string",
                    "enum": ["celsius", "fahrenheit"],
                    "description": "Temperatur-Einheit.
Default: celsius"
                }
            },
            "required": ["city"]
        }
    },
    {
        "name": "search_database",
        "description": "Durchsucht die Produktdatenbank
nach Artikeln.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": "Suchbegriff"
                },
                "category": {
                    "type": "string",
                    "enum": ["electronics", "books", "clo-
thing"],
                    "description": "Kategorie-Filter (opti-
onal)"
                }
            }
        }
    }
]

```

```

        },
        "max_results": {
            "type": "integer",
            "description": "Maximale Anzahl Ergeb-
nisse (default: 5)",
            "default": 5
        }
    },
    "required": ["query"]
}
]

```

Der Tool-Use-Loop

```

import json

def process_tool_call(tool_name: str, tool_input: dict) ->
str:
    """Führe das Tool aus und gib das Ergebnis zurück."""
    if tool_name == "get_weather":
        # Echte API-Call hier
        return json.dumps({
            "city": tool_input["city"],
            "temperature": 12,
            "condition": "bewölkt",
            "humidity": 65
        })
    elif tool_name == "search_database":
        # Echte DB-Query hier
        return json.dumps({
            "results": [
                {"name": "Laptop X", "price": 999},
                {"name": "Tablet Y", "price": 499}
            ],
            "total": 2
        })
    else:
        return json.dumps({"error": f"Unknown tool: {tool_n
ame}"})

def chat_with_tools(user_message: str) -> str:
    messages = [{"role": "user", "content": user_message}]

    while True:
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=1024,
            tools=tools,
            messages=messages
        )

        # Wenn das Modell fertig ist (keine Tool-Calls
mehr)
        if response.stop_reason == "end_turn":
            return next(
                block.text for block in response.content
                if block.type == "text"
            )

```

```

# Tool-Calls verarbeiten
tool_results = []
for block in response.content:
    if block.type == "tool_use":
        result = process_tool_call(block.name, block.input)

        tool_results.append({
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": result
        })

# Antwort + Tool-Ergebnisse anhängen
messages.append({"role": "assistant", "content": response.content})
messages.append({"role": "user", "content": tool_results})

# Nutzung
answer = chat_with_tools("Wie ist das Wetter in Berlin und München?")
# Das Modell ruft get_weather ZWEIMAL auf (einmal pro Stadt)
# und fasst die Ergebnisse zusammen.

```

Tool-Design: Best Practices

1. Klare, präzise Beschreibungen

```

# SCHLECHT:
{"name": "search", "description": "Sucht nach Sachen"}

# GUT:
{"name": "search_products",
 "description": "Durchsucht den Produktkatalog nach Artikeln. "

"Gibt Name, Preis und Verfügbarkeit zurück. "
      "Nutze dieses Tool wenn der User nach
Produkten, "
      "Preisen oder Verfügbarkeit fragt."}

```

2. Sinnvolle Parameter-Beschreibungen

```
# SCHLECHT:
"query": {"type": "string"}

# GUT:
"query": {
  "type": "string",
  "description": "Suchbegriff. Kann Produktname,
  Kategorie oder "
                "Beschreibung sein. Beispiele: 'rotes
  Kleid', 'iPhone 16', 'Geschenk unter 50€'"
}
```

3. Enums für begrenzte Optionen

```
"status": {
  "type": "string",
  "enum": ["active", "inactive", "pending"],
  "description": "Filtere nach Status"
}
```

4. Wenige, mächtige Tools statt vieler kleiner

```
# SCHLECHT: 10 einzelne Tools
"get_user_name", "get_user_email", "get_user_address", ...

# GUT: 1 flexibles Tool
"get_user_info": {
  "fields": {
    "type": "array",
    "items": {"enum": ["name", "email", "address", "phone"]},
    "description": "Welche Felder abgerufen werden sollen"
  }
}
```

Parallele Tool-Calls

Claude und GPT können mehrere Tools gleichzeitig aufrufen:

User: "Vergleiche das Wetter in Berlin, München und Hamburg."

LLM generiert 3 parallele Tool-Calls:

```
get_weather(city="Berlin")  
get_weather(city="München")  
get_weather(city="Hamburg")
```

- Alle 3 werden gleichzeitig ausgeführt
- LLM bekommt alle 3 Ergebnisse auf einmal
- Fasst in einer Antwort zusammen

Sicherheit bei Tool Use

Bestätigungen für kritische Aktionen

```
# Tool-Definition mit Sicherheitsstufe
critical_tools = {
    "delete_user": "CRITICAL",      # Immer bestätigen
    "send_email": "CONFIRM",        # User fragen
    "search_database": "SAFE",      # Automatisch ausführen
}

def process_tool_call_safe(tool_name, tool_input):
    level = critical_tools.get(tool_name, "CONFIRM")

    if level == "CRITICAL":
        raise PermissionError(f"Tool
'{tool_name}' benötigt Admin-Bestätigung")
    elif level == "CONFIRM":
        # User um Bestätigung bitten
        print(f"Tool: {tool_name}, Input: {tool_input}")
        if input("Ausführen? (y/n): ") != "y":
            return '{"status": "cancelled_by_user"}'

    return process_tool_call(tool_name, tool_input)
```


Input-Validierung

```
from pydantic import BaseModel, validator

class WeatherInput(BaseModel):
    city: str
    unit: str = "celsius"

    @validator("city")
    def validate_city(cls, v):
        # Keine SQL-Injection, keine Pfadangaben
        if any(c in v for c in [";", "'", "/", "\\"]):
            raise ValueError("Ungültiger Stadtname")
        return v

    @validator("unit")
    def validate_unit(cls, v):
        if v not in ["celsius", "fahrenheit"]:
            raise ValueError("Ungültige Einheit")
        return v
```

Tool Use in der Praxis

Beispiel: Kunden-Support-Bot

```

support_tools = [
    {
        "name": "lookup_order",
        "description": "Suche eine Bestellung nach Bestell-
nummer oder Kundennummer.",
        "input_schema": {
            "type": "object",
            "properties": {
                "order_id": {"type": "string"},
                "customer_id": {"type": "string"}
            }
        }
    },
    {
        "name": "check_inventory",
        "description": "Prüfe die Verfügbarkeit eines Pro-
dukts.",
        "input_schema": {
            "type": "object",
            "properties": {
                "product_id": {"type": "string"},
                "quantity": {"type": "integer", "default":
1}
            },
            "required": ["product_id"]
        }
    },
    {
        "name": "create_ticket",
        "description": "Erstelle ein Support-Ticket für
komplexe Anliegen.",
        "input_schema": {
            "type": "object",
            "properties": {
                "subject": {"type": "string"},
                "description": {"type": "string"},
                "priority": {"type": "string", "enum": ["lo
w", "medium", "high"]},
                "customer_id": {"type": "string"}
            },
            "required": ["subject", "description", "priori
ty"]
        }
    }
]

```

Übungen

Übung 1: Eigenes Tool

Definiere ein Tool für einen Anwendungsfall deiner Wahl. Implementiere den Tool-Use-Loop.

Übung 2: Multi-Tool

Erstelle 3 Tools, die zusammenarbeiten (z.B. Suche → Details → Bestellung). Lass das Modell sie in der richtigen Reihenfolge aufrufen.

Übung 3: Sicherheit

Implementiere eine Sicherheitsschicht, die kritische Tool-Calls blockiert oder bestätigen lässt.

Übung 4: Parallele Calls

Teste parallele Tool-Calls: Lass das Modell 5 verschiedene Informationen gleichzeitig abrufen.

Kapitel 7: Agentische Systeme

– KI, die handelt

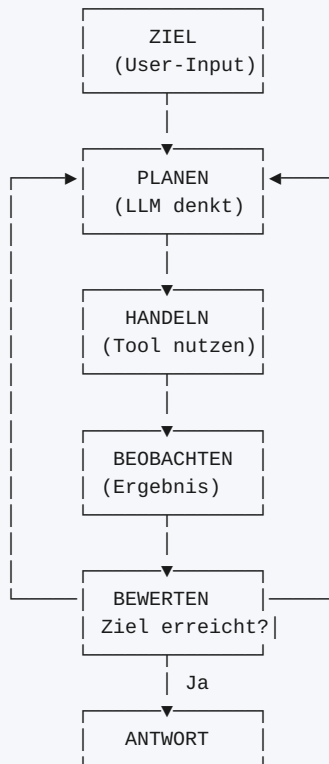
Tool Use (Kapitel 6) gibt KI Hände. Agentische Systeme geben ihr einen Plan.

Ein Agent ist ein LLM, das:

1. Ein Ziel bekommt
2. Einen Plan erstellt
3. Tools nutzt, um Schritte auszuführen
4. Ergebnisse bewertet
5. Den Plan anpasst
6. Iteriert, bis das Ziel erreicht ist

Der Unterschied zu einem einfachen Tool-Use-Call: Ein Agent arbeitet autonom über mehrere Schritte. Er entscheidet selbst, welches Tool als Nächstes kommt.

Agent-Architektur



Der einfachste Agent: ReAct-Loop

```

import anthropic
import json

client = anthropic.Anthropic()

SYSTEM_PROMPT = ""Du bist ein hilfreicher Agent.
Du hast Zugang zu Tools, um Aufgaben zu erledigen.

Arbeite Schritt für Schritt:
1. Überlege, was du als Nächstes tun musst
2. Nutze ein Tool
3. Beobachte das Ergebnis
4. Entscheide: Bist du fertig oder brauchst du mehr Schritte?

Wenn du die Aufgabe erledigt hast, antworte dem User direkt.""

def run_agent(user_task: str, tools: list, max_steps: int
= 10) -> str:
    messages = [{"role": "user", "content": user_task}]

    for step in range(max_steps):
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=4096,
            system=SYSTEM_PROMPT,
            tools=tools,
            messages=messages
        )

        # Wenn keine Tool-Calls → Agent ist fertig
        if response.stop_reason == "end_turn":
            return next(
                block.text for block in response.content
                if block.type == "text"
            )

        # Tool-Calls ausführen
        messages.append({"role": "assistant", "content": response.content})

        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                print(f" Schritt {step+1}: {block.name}({j

```



```

son.dumps(block.input, ensure_ascii=False)[:100]})")
        result = execute_tool(block.name, block.in-
put)

        tool_results.append({
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": result
        })

        messages.append({"role": "user", "content": tool_re
sults})

        return "Agent hat das Schritt-Limit erreicht."

```

MCP – Model Context Protocol

MCP (Model Context Protocol) ist der offene Standard für die Verbindung von Agenten mit externen Systemen. Im November 2024 von Anthropic eingeführt, im Dezember 2025 an die Linux Foundation übergeben. Stand März 2026: 97 Millionen monatliche SDK-Downloads, adoptiert von Anthropic, OpenAI, Google, Microsoft und Amazon.

Was MCP löst

Ohne MCP:

```

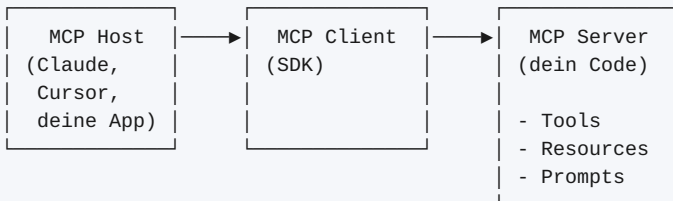
Agent —> Custom Code —> Datenbank
Agent —> Custom Code —> Dateisystem
Agent —> Custom Code —> GitHub API
Agent —> Custom Code —> Slack API

```

Mit MCP:

Agent —> MCP Client —> MCP Server (DB)
—> MCP Server (Dateien)
—> MCP Server (GitHub)
—> MCP Server (Slack)

MCP-Architektur



Einen MCP-Server schreiben (Python)

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Mein Firmen-MCP-Server")

@mcp.tool()
def search_knowledge_base(query: str, max_results: int = 5) -> str:
    """Durchsucht die interne Wissensdatenbank des Unternehmens.
    Nutze dieses Tool für Fragen zu Firmenrichtlinien,
    Prozessen und internem Wissen."""
    results = db.search(query, limit=max_results)
    return json.dumps(results)

@mcp.tool()
def create_jira_ticket(
    title: str,
    description: str,
    priority: str = "medium",
    assignee: str | None = None
) -> str:
    """Erstellt ein Jira-Ticket. Nutze dieses Tool wenn der
    User
    ein Ticket, Issue oder Task erstellen möchte."""
    ticket = jira_client.create_issue(
        project="PROJ",
        summary=title,
        description=description,
        priority=priority,
        assignee=assignee
    )
    return json.dumps({"ticket_id": ticket.key, "url": ticket.url})

@mcp.resource("file://docs/{path}")
def read_document(path: str) -> str:
    """Liest ein Dokument aus dem Docs-Verzeichnis."""
    return (Path("docs") / path).read_text()

# Server starten
mcp.run()
```

MCP-Server nutzen (in Claude Code)

```
// .claude/settings.json oder ~/.claude.json
{
  "mcpServers": {
    "firmen-kb": {
      "command": "python",
      "args": ["mcp_server.py"],
      "env": {
        "DB_URL": "postgresql://..."
      }
    }
  }
}
```

Jetzt kann Claude Code automatisch deine Wissensdatenbank durchsuchen und Jira-Tickets erstellen.

Multi-Agent-Systeme

Für komplexe Aufgaben: Mehrere spezialisierte Agenten, die zusammenarbeiten.

Orchestrator-Pattern

```
async def orchestrator(task: str) -> str:
    """Ein Orchestrator delegiert an spezialisierte Agen-
    ten."""

    # Schritt 1: Aufgabe analysieren und Plan erstellen
    plan = await planning_agent(task)

    # Schritt 2: Teilaufgaben an Spezialisten delegieren
    results = {}
    for subtask in plan.subtasks:
        if subtask.type == "research":
            results[subtask.id] = await research_agent(sub-
task)
        elif subtask.type == "code":
            results[subtask.id] = await coding_agent(sub-
task)
        elif subtask.type == "review":
            results[subtask.id] = await review_agent(sub-
task)

    # Schritt 3: Ergebnisse zusammenführen
    final = await synthesis_agent(task, results)
    return final
```

Supervisor-Pattern

```
async def supervisor_loop(task: str, agents: dict) -> str:
    """Ein Supervisor wählt den nächsten Agent und
    überwacht den Fortschritt."""
    context = {"task": task, "history": [], "status": "in_p
    rogress"}

    while context["status"] != "done":
        # Supervisor entscheidet, welcher Agent als
        Nächstes dran ist
        decision = await supervisor_decide(context, list(ag
        ents.keys()))

        if decision.action == "delegate":
            result = await agents[decision.agent]
            (decision.subtask, context)
            context["history"].append({
                "agent": decision.agent,
                "task": decision.subtask,
                "result": result
            })

        elif decision.action == "done":
            context["status"] = "done"

    return await compile_final_answer(context)
```

Anthropic Agent SDK

Anthropics SDK für den Bau produktionsreifer Agenten:

```

from claude_agent_sdk import Agent, tool

class ResearchAgent(Agent):
    """Agent für Recherche-Aufgaben."""

    model = "claude-sonnet-4-6"
    system_prompt = """Du bist ein Recherche-Agent.
    Du suchst nach Informationen und fasst sie zusammen.
    Nutze die verfügbaren Tools systematisch."""

    @tool
    def web_search(self, query: str) -> str:
        """Suche im Web nach Informationen."""
        return search_api.search(query)

    @tool
    def read_url(self, url: str) -> str:
        """Lese den Inhalt einer Webseite."""
        return fetch_and_extract(url)

# Agent ausführen
agent = ResearchAgent()
result = agent.run("Recherchiere die neuesten Trends in er-
neuerbaren Energien 2026")

```

Agent-Guardrails

Agenten, die autonom handeln, brauchen Sicherheitsmechanismen:

1. Token-Budget

```
MAX_TOKENS_PER_TASK = 100_000 # Kosten-Limit

def run_agent_with_budget(task, budget=MAX_TOKENS_PER_TASK)
:
    total_tokens = 0
    while total_tokens < budget:
        response = call_llm(...)
        total_tokens += response.usage.input_tokens + re-
sponse.usage.output_tokens
        if is_done(response):
            return response
        raise BudgetExceededError(f"Token-Budget ({budget})
überschritten")
```

2. Erlaubte Aktionen

```
ALLOWED_ACTIONS = {
    "read_file": True,          # Immer erlaubt
    "write_file": "confirm",    # User muss bestätigen
    "delete_file": False,       # Nie erlaubt
    "run_command": "confirm",
    "send_email": False,
    "query_database": True,
    "modify_database": "confirm",
}
```

3. Sandbox

Agenten sollten in isolierten Umgebungen laufen:

- Docker-Container für Code-Ausführung
- Read-only Dateisysteme wo möglich
- Netzwerk-Beschränkungen
- Zeitlimits für einzelne Operationen

4. Human-in-the-Loop

Für kritische Entscheidungen: Pausiere und frage den Menschen:

```
def agent_step(action):
    if action.risk_level == "high":
        approved = ask_human(f"Agent möchte: {action.description}\nErlauben?")
        if not approved:
            return "Aktion abgelehnt"
    return execute(action)
```

Wann Agenten, wann nicht?

Situation	Agent?	Warum
Klar definierte Aufgabe, 1 Schritt	Nein	Einfacher API-Call reicht
Multi-Step mit klarer Reihenfolge	Prompt-Chain	Deterministische Pipeline reicht
Multi-Step mit unklarer Reihenfolge	Ja	Agent entscheidet dynamisch
Offene Recherche-Aufgabe	Ja	Agent iteriert bis zufrieden
Sicherheitskritische Aufgabe	Vorsichtig	Starke Guardrails nötig
Echtzeit / Low-Latency	Nein	Agenten sind langsam (mehrere LLM-Calls)

Übungen

Übung 1: ReAct-Agent

Baue einen einfachen ReAct-Agent mit 2-3 Tools (z.B. Web-Suche, Rechner, Wetter). Lass ihn eine mehrstufige Frage beantworten.

Übung 2: MCP-Server

Schreibe einen MCP-Server für einen Anwendungsfall deiner Wahl (z.B. Zugriff auf ein lokales Dateisystem, eine SQLite-DB oder eine API).

Übung 3: Guardrails

Implementiere Token-Budget, erlaubte Aktionen und Human-in-the-Loop für deinen Agent. Teste: Hält er sich daran?

Übung 4: Multi-Agent

Baue ein 2-Agent-System: Ein Agent recherchiert, der andere schreibt basierend auf der Recherche. Koordiniere sie mit einem Orchestrator.

Kapitel 8: Fine-Tuning vs. Prompting – Die Entscheidungsmatrix

“Sollte ich das Modell fine-tunen?” Diese Frage höre ich ständig. Und die Antwort ist fast immer: Nein. Zumindest nicht zuerst.

Fine-Tuning ist mächtig. Aber es ist teuer, komplex und oft unnötig. In den meisten Fällen erreichst du mit gutem Prompting, RAG und Tool Use dasselbe Ergebnis – schneller und günstiger.

Dieses Kapitel zeigt dir, wann welcher Ansatz der richtige ist.

Die Optionen im Überblick

EINFACHHEIT		KOMPLEXITÄT		
▼			▼	
Prompt Engineering trainieren	Few-Shot Prompting	RAG Pipeline	Fine-Tuning (bestehend)	Eigenes Modell
Minuten	Stunden	Tage	Wochen	Monate
\$0	\$0.01	\$100	\$1.000+	\$100.000+

Wann reicht Prompting?

Anforderung	Prompting reicht?
Ton/Stil anpassen	✓ Ja – System-Prompt mit Beispielen
Fachsprache verwenden	✓ Ja – Fachbegriffe im Prompt erklären
Ausgabe-Format steuern	✓ Ja – Structured Output / Tool Use
Auf eigene Daten zugreifen	✓ Ja – RAG
Eigene Tools nutzen	✓ Ja – Function Calling
Komplexe Reasoning-Aufgaben	✓ Ja – Chain-of-Thought, Extended Thinking
95%+ Accuracy bei Klassifikation	⚠ Vielleicht – Few-Shot oft ausreichend
Spezifische Domäne (Jura, Medizin)	⚠ Vielleicht – RAG + spezialisierter Prompt
Eigene “Stimme” / Brand Voice	⚠ Vielleicht – System-Prompt + Beispiele
Extrem hohe Konsistenz	✗ Vielleicht Fine-Tuning nötig
Latenz-kritisch (jede ms zählt)	✗ Fine-Tuning auf kleinem Modell
Offline-Betrieb	✗ Eigenes/lokales Modell nötig

Wann Fine-Tuning?

Fine-Tuning ist sinnvoll, wenn:

1. **Konsistenz über tausende Anfragen** – Du brauchst exakt dasselbe Format, denselben Stil, immer. Nicht “meistens”, sondern “immer”.

2. **Latenz** – Du willst ein kleines, schnelles Modell, das eine spezifische Aufgabe so gut kann wie ein großes.
3. **Kosten bei Volumen** – 1 Million Requests/Tag: Ein fine-getuntes GPT-4o-mini ist günstiger als ein Prompt mit 2000 Token Kontext bei Sonnet.
4. **Spezifisches Verhalten** – Das Modell soll Dinge tun, die schwer per Prompt zu beschreiben sind (z.B. einen sehr spezifischen Coding-Style).
5. **Datenschutz** – Deine Trainingsdaten bleiben bei dir (lokales Fine-Tuning mit Open Source).

Fine-Tuning-Optionen (Stand 2026)

Cloud-basiert

Anbieter	Modell	Methode	Kosten (Training)	Kosten (Inference)
OpenAI	GPT-4o-mini	Supervised FT	\$3.00 / 1M Token	\$0.30 / 1M Token (out)
OpenAI	GPT-4o	Supervised FT	\$25.00 / 1M Token	\$3.75 / 1M Token (out)
Google	Gemini Flash	Supervised FT	Variabel	Reduziert vs. Base
Together AI	Llama, Mistral etc.	LoRA, Full FT	Ab \$0.50/Stunde GPU	Pay-per-token
Fireworks AI	Diverse Open Source	LoRA	Ab \$0.50/Stunde GPU	Pay-per-token

Lokal

Tool	Methode	Stärke
Unsloth	QLoRA	2x schneller, 60% weniger VRAM, Open Source
Axolotl	LoRA, QLoRA, Full FT	Flexibel, viele Modelle
Hugging Face TRL	SFT, DPO, RLHF	Standard-Framework
MLX (Apple)	LoRA	Optimiert für Apple Silicon
Ollama + Modelfile	Basis-Anpassung	Einfachster Einstieg

Fine-Tuning: Der Prozess

Schritt 1: Trainingsdaten erstellen

```
// Format: JSONL (eine Zeile pro Beispiel)
{"messages": [
  {"role": "system", "content": "Du bist ein Support-Bot für TechCorp."},
  {"role": "user", "content": "Wie setze ich mein Passwort zurück?"},
  {"role": "assistant", "content": "Gehe auf techcorp.de/passwort, klicke auf 'Passwort vergessen', gib deine E-Mail ein. Du erhältst innerhalb von 5 Minuten einen Reset-Link."}
]}
{"messages": [
  {"role": "system", "content": "Du bist ein Support-Bot für TechCorp."},
  {"role": "user", "content": "Wie kündige ich mein Abo?"},
  {"role": "assistant", "content": "Gehe zu Einstellungen > Abo > Kündigen. Die Kündigung wird zum Ende des aktuellen Abrechnungszeitraums wirksam. Du behältst bis dahin vollen Zugang."}
]}
```

Wie viele Beispiele?

Aufgabe	Minimum	Empfohlen	Optimum
Stil/Ton anpassen	20	50-100	200+
Klassifikation	50 pro Klasse	100 pro Klasse	500+ pro Klasse
Domänen-spezifisch	100	500	1.000+
Komplexe Aufgaben	200	1.000	5.000+

Schritt 2: Datenqualität prüfen

- # Checkliste für Trainingsdaten:
- # ☒ Konsistente Formatierung
- # ☒ Keine Widersprüche zwischen Beispielen
- # ☒ Repräsentativ für echte Anfragen
- # ☒ Korrekte Antworten (manuell geprüft)
- # ☒ Diverse Formulierungen (nicht alle gleich)
- # ☒ Edge Cases enthalten
- # ☒ Keine sensiblen Daten (PII, Passwörter)

Schritt 3: Fine-Tuning starten (OpenAI Beispiel)

```
from openai import OpenAI
client = OpenAI()

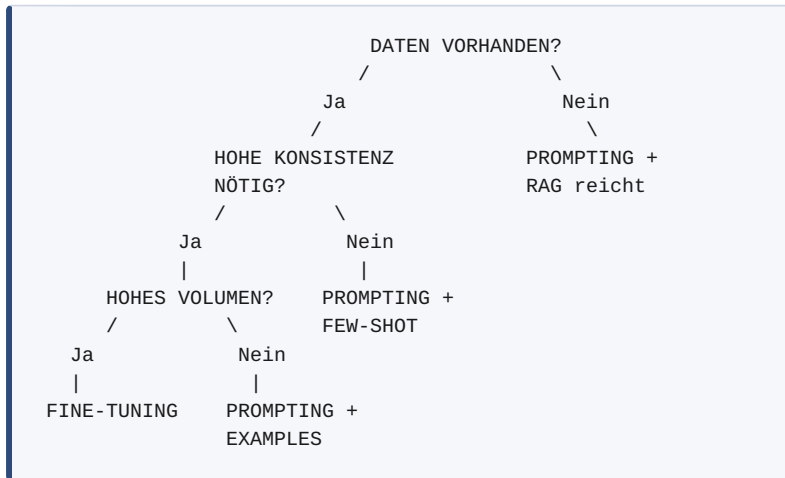
# Trainingsdatei hochladen
file = client.files.create(
    file=open("training_data.jsonl", "rb"),
    purpose="fine-tune"
)

# Fine-Tuning starten
job = client.fine_tuning.jobs.create(
    training_file=file.id,
    model="gpt-4o-mini-2024-07-18",
    hyperparameters={
        "n_epochs": 3,
        "batch_size": "auto",
        "learning_rate_multiplier": "auto"
    }
)

# Status prüfen
status = client.fine_tuning.jobs.retrieve(job.id)
print(f"Status: {status.status}")
# → "running" → "succeeded"

# Fine-tuned Modell nutzen
response = client.chat.completions.create(
    model=status.fine_tuned_model, # ft:gpt-4o-
    mini:org:custom-name:id
    messages=[{"role": "user", "content": "Wie setze ich
    mein Passwort zurück?"}]
)
```

Die Entscheidungsmatrix



Praktische Faustregel

Starte IMMER mit Prompting. Investiere zuerst 2-3 Tage in Prompt-Optimierung, bevor du über Fine-Tuning nachdenkst. In 90% der Fälle wirst du feststellen: Prompting reicht.

Wenn es nicht reicht, probiere diese Reihenfolge:

1. Besserer System-Prompt
2. Few-Shot-Beispiele im Prompt
3. RAG (eigene Daten)
4. Prompt-Chaining
5. Fine-Tuning (wenn 1-4 nicht reichen)

Prompt-Distillation: Der goldene Mittelweg

Eine clevere Technik: Nutze ein großes, teures Modell (Opus) um hochqualitative Antworten zu generieren. Nutze diese Antworten als Trainingsdaten für ein kleines, günstiges Modell (Haiku, GPT-4o-mini).

```
# Phase 1: Großes Modell generiert Trainingsbeispiele
for query in production_queries:
    response = opus_model.generate(query) # Teuer, langsam, aber gut
    training_data.append({"input": query, "output": response})

# Phase 2: Kleines Modell wird auf diesen Daten fine-tuned
fine_tune(small_model, training_data)

# Phase 3: Kleines Modell in Produktion (günstig, schnell)
result = fine_tuned_small_model.generate(new_query)
```

Ergebnis: 90% der Qualität des großen Modells bei 10% der Kosten und 5x der Geschwindigkeit.

Übungen

Übung 1: Prompt vs. Fine-Tuning

Wähle eine Aufgabe (z.B. E-Mail-Klassifikation) und löse sie mit Few-Shot-Prompting. Wie hoch ist die Accuracy? Würde Fine-Tuning helfen?

Übung 2: Trainingsdaten erstellen

Erstelle 20 Trainingsdaten-Paare für eine Aufgabe deiner Wahl. Prüfe sie auf Konsistenz und Qualität.

Übung 3: Kosten-Analyse

Berechne die Kosten für: 100.000 Anfragen/Monat, einmal mit Sonnet (mit langem System-Prompt) und einmal mit fine-tuned GPT-4o-mini (ohne System-Prompt). Was ist günstiger?

Übung 4: Distillation

Wenn du API-Zugang hast: Generiere 50 Antworten mit einem großen Modell. Nutze sie als Few-Shot-Beispiele für ein kleines Modell. Wie nah kommt das kleine Modell?

Kapitel 9: Context Engineering

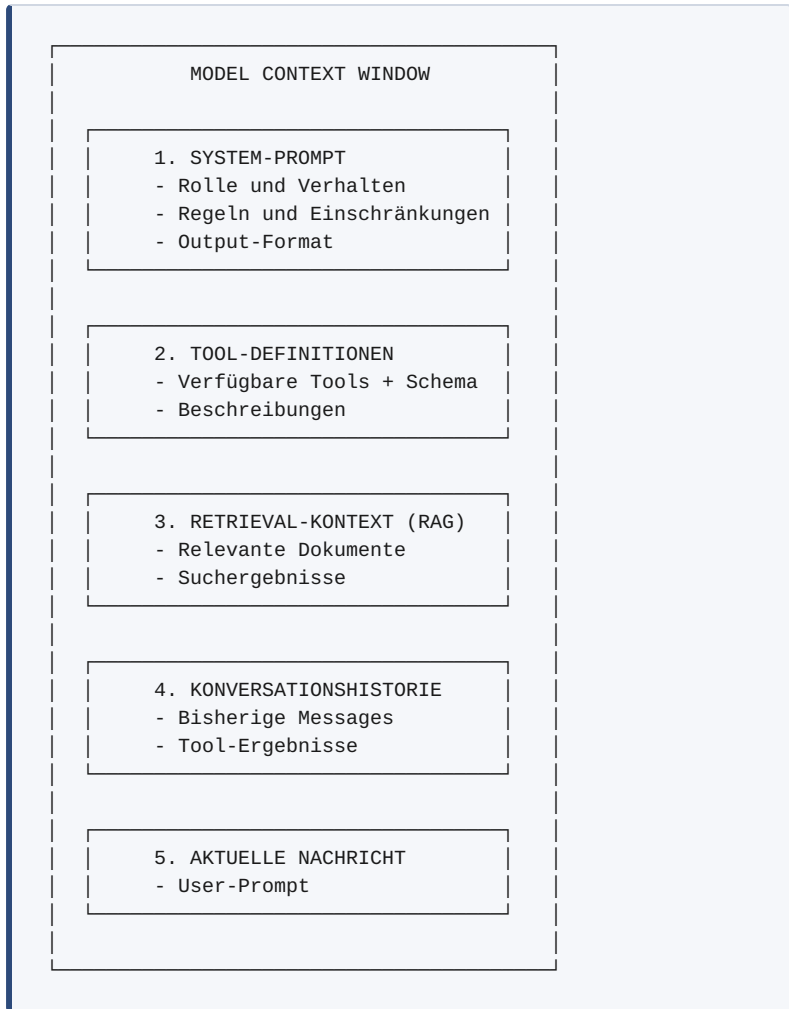
– Die Evolution des Prompt Engineering

Prompt Engineering war Band 1. Context Engineering ist Band 7.

Der Unterschied: Prompt Engineering optimiert den Text, den du an ein LLM schickst. Context Engineering optimiert *alles*, was das Modell sieht – System-Prompt, Tools, Retrieval-Ergebnisse, Konversationshistorie, Caching, und wie all diese Teile zusammenspielen.

Was ist Context Engineering?

Jeder API-Call an ein LLM hat mehrere Kontextschichten:



Context Engineering optimiert jede dieser Schichten – und ihr Zusammenspiel.

Schicht 1: System-Prompt Engineering

Die Anatomie eines Produktions-System-Prompts

```
SYSTEM_PROMPT = ""  
# Rolle  
Du bist der Kundenservice-Assistent von TechCorp.  
Du hilfst Kunden mit Fragen zu unseren Produkten und Services.  
  
# Verhalten  
- Antworte auf Deutsch  
- Duze die Kunden  
- Sei freundlich, aber effizient – keine unnötigen Floskeln  
- Wenn du die Antwort nicht weißt: Sage es ehrlich und leite an den menschlichen Support weiter  
  
# Fähigkeiten  
- Du kannst Bestellungen nachschlagen (Tool: lookup_order)  
- Du kannst den Bestellstatus prüfen (Tool: check_status)  
- Du kannst Rückgaben einleiten (Tool: initiate_return)  
- Du kannst KEINE Zahlungen verarbeiten oder Preise ändern  
  
# Einschränkungen  
- Gib NIEMALS interne Systeminfos preis  
- Diskutiere NICHT über Wettbewerber  
- Versprich KEINE Rabatte oder Sonderkonditionen  
- Bei technischen Problemen: Erstelle ein Ticket, versuche nicht selbst zu debuggen  
  
# Kontext  
- Heute ist {date}  
- Aktuelle Promotions: {current_promotions}  
- Bekannte Probleme: {known_issues}  
  
# Output-Format  
- Kurze, klare Antworten (max. 3 Absätze)  
- Bei Schritt-für-Schritt-Anleitungen: Nummerierte Liste  
- Bei mehreren Optionen: Bullet Points  
""
```

System-Prompt-Patterns

Pattern 1: Identity + Constraints

```
Wer bist du? → Rolle, Name, Expertise  
Was kannst du? → Fähigkeiten, Tools  
Was kannst du NICHT? → Grenzen, Verbote
```

Pattern 2: Dynamic Context

```
# Kontext, der sich bei jedem Call ändert  
system = f"""  
Aktuelles Datum: {datetime.now().strftime('%d.%m.%Y')}  
Benutzer: {user.name} (Kunde seit {user.since}, Plan:  
{user.plan})  
Letzte Interaktion: {user.last_interaction}  
Offene Tickets: {user.open_tickets}  
"""
```

Pattern 3: Graduated Response

```
Für einfache Fragen: Antworte direkt (1-2 Sätze)  
Für mittlere Fragen: Antworte mit Erklärung (1 Absatz)  
Für komplexe Fragen: Antworte strukturiert (Überschriften,  
Listen)  
Für Fragen außerhalb deines Wissens: Leite weiter
```

Schicht 2: Tool-Kontext-Optimierung

Die Beschreibungen deiner Tools sind Teil des Kontexts – und sie beeinflussen stark, wie gut das Modell die Tools nutzt.

Schlechte Tool-Beschreibung

```
"name": "search"  
"description": "Sucht Dinge"
```

→ Modell weiß nicht, wann es das Tool nutzen soll.

Gute Tool-Beschreibung

```
"name": "search_knowledge_base"  
"description": "Durchsucht die interne Wissensdatenbank von  
TechCorp.  
Enthält: Produktdokumentation, FAQ, Troubleshooting-Guides,  
Prozessbeschreibungen.  
Enthält NICHT: Bestelldaten, Kundendaten, Finanzdaten.  
Nutze dieses Tool BEVOR du eine Frage beantwortest, die  
spezifisches Produktwissen erfordert.  
Beispiel-Queries: 'Rückgabe einleiten', 'Garantiebedingun-  
gen Laptop Pro', 'Passwort zurücksetzen'"
```

Schicht 3: RAG-Kontext-Optimierung

Kontext-Reihenfolge

Die Reihenfolge der Chunks im Kontext beeinflusst die Qualität:

```

# Am relevantesten zuerst (Primacy Bias)
# ODER am relevantesten zuletzt (Recency Bias)
# Empfehlung: Am relevantesten zuerst UND zuletzt

def arrange_context(chunks, query):
    # Sortiere nach Relevanz
    ranked = sorted(chunks, key=lambda c: c.similarity, reverse=True)

    if len(ranked) <= 3:
        return ranked

    # Bestes am Anfang, zweitbestes am Ende
    return [ranked[0]] + ranked[2:] + [ranked[1]]

```

Kontext-Kompression

Zu viel Kontext ist schlechter als zu wenig:

```

# Vor dem Einfügen in den Prompt: Zusammenfassen
summary = client.messages.create(
    model="claude-haiku-3-5-20241022",
    max_tokens=200,
    messages=[{
        "role": "user",
        "content": f"Fasse die für diese Frage relevanten
Informationen zusammen:\n\nFrage: {question}\n\nDokument:
\n{chunk}"
    }]
).content[0].text

```

Schicht 4: Konversationshistorie managen

Problem: Kontext-Fenster voll

Lange Konversationen sprengen das Kontext-Fenster. Strategien:

```

class ConversationManager:
    def __init__(self, max_messages=20, max_tokens=50000):
        self.messages = []
        self.max_messages = max_messages
        self.max_tokens = max_tokens

    def add_and_trim(self, message):
        self.messages.append(message)

        # Strategie 1: Älteste entfernen
        while len(self.messages) > self.max_messages:
            self.messages.pop(1) # Index 0 = System, be-
halten

        # Strategie 2: Zusammenfassen statt entfernen
        if self.estimate_tokens() > self.max_tokens:
            self._summarize_old_messages()

    def _summarize_old_messages(self):
        old = self.messages[1:-4] # Mittlere Messages
        summary = summarize_conversation(old)
        self.messages = (
            [self.messages[0]] # System
            + [{"role": "user", "content": f"[Zusammenfas-
sung bisheriger Konversation: {summary}]"}]
            + self.messages[-4:] # Letzte 4 Messages
        )

```

Schicht 5: Prompt Caching

Anthropics Prompt Caching ist eine der mächtigsten Optimierungen:

```

# Ohne Caching: 200K Token System-Prompt = teuer bei JEDEM
Call
# Mit Caching: Erster Call zahlt voll, alle weiteren 90%
günstiger

message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system=[
        {
            "type": "text",
            "text": LONG_SYSTEM_PROMPT, # z.B. 50.000 To-
ken
            "cache_control": {"type": "ephemeral"}
        }
    ],
    messages=[{"role": "user", "content": user_question}]
)

# Cache-Statistiken prüfen
print(f"Cache Write: {message.usage.cache_creation_input_tokens}")
print(f"Cache Read: {message.usage.cache_read_input_tokens}")
# Erster Call: Cache Write = 50.000, Cache Read = 0
# Zweiter Call: Cache Write = 0, Cache Read = 50.000 (90%
günstiger!)

```

Was cachen?

Kontext-Teil	Cachen?	Warum
System-Prompt	✓ Immer	Ändert sich selten
Tool-Definitionen	✓ Immer	Ändert sich nie
RAG-Dokumente	⚠ Manchmal	Nur wenn wiederholt dieselben Docs
Konversations-historie	✓ Ja	Prefix bleibt gleich, nur neue Messages kommen dazu
User-Prompt	✗ Nein	Ändert sich bei jedem Call

Context Engineering Checkliste

Für jeden produktionsreifen LLM-Call prüfe:

System-Prompt

- [] Rolle klar definiert?
- [] Fähigkeiten und Grenzen explizit?
- [] Dynamischer Kontext (Datum, User-Info) eingefügt?
- [] Output-Format spezifiziert?
- [] Gecacht?

Tools

- [] Beschreibungen detailliert genug?
- [] Beispiele in den Beschreibungen?
- [] Negative Beispiele (“Nutze dieses Tool NICHT für...”)?

- [] Gecacht?

RAG

- [] Relevante Chunks ausgewählt (nicht zu viele)?
- [] Reranking angewandt?
- [] Quellen referenzierbar?
- [] Kontext-Reihenfolge optimiert?

History

- [] Alte Messages zusammengefasst oder getrimmt?
- [] Token-Budget eingehalten?
- [] Prefix gecacht?

Prompt

- [] Klar und eindeutig?
- [] Alle nötigen Informationen enthalten?
- [] Nicht zu lang?

Von Prompt Engineering zu Context Engineering

Prompt Engineering	Context Engineering
“Wie schreibe ich den Prompt?”	“Wie designe ich den gesamten Kontext?”
Einzelner Text-Input	System + Tools + RAG + History + Prompt
Statisch	Dynamisch (ändert sich pro User, pro Anfrage)
Trial and Error	Systematisch, messbar, versioniert
Ein Skill	Eine Disziplin

Context Engineering ist die Zukunft. Wer nur Prompts schreibt, schöpft 20% des Potenzials aus. Wer den gesamten Kontext designt, schöpft 100% aus.

Übungen

Übung 1: System-Prompt Audit

Nimm einen bestehenden System-Prompt und prüfe ihn gegen die Checkliste. Was fehlt?

Übung 2: Caching implementieren

Implementiere Prompt Caching für einen Anwendungsfall mit langem System-Prompt. Miss den Kostenunterschied.

Übung 3: Konversationsmanager

Baue einen ConversationManager mit Zusammenfassungs-Strategie. Teste: Behält das Modell den Kontext?

Übung 4: Full Context Design

Designe den kompletten Kontext für einen produktionsreifen Chatbot: System-Prompt, Tools, RAG-Integration, History-Management, Caching. Dokumentiere jede Entscheidung.

Kapitel 10: Zusammenfassung und Ausblick

Sieben Bände. Du bist jetzt in der Profi-Liga.

In diesem Band hast du den Sprung gemacht: Vom Chat-Interface zur API. Vom Endnutzer zum Entwickler. Vom einzelnen Prompt zum System.

Was du jetzt kannst

1. **Code-Generierung** – Du schreibst Prompts, die professionellen Code erzeugen: mit Kontext, Architektur, Qualitätsanforderungen und Testabdeckung. Du kennst die Unterschiede zwischen Feature-Implementierung, Bug-Fixing, Refactoring und Code Reviews.
2. **Agentic Coding Tools** – Du kennst die Landschaft: Claude Code, Cursor, GitHub Copilot, Windsurf, Cline, Aider. Du weißt, wie CLAUDE.md und .cursorrules funktionieren. Du promptest innerhalb dieser Tools optimal.
3. **LLM-APIs** – Du beherrschst die APIs von Anthropic, OpenAI und Google. Du kennst Streaming, Structured Output, Vision, Prompt Caching und Batches. Du weißt, was jedes Modell kostet und wann du welches nutzt.
4. **Programmatisches Prompting** – Du baust Prompt-Templates, Multi-Turn-Konversationen, Fehlerbehandlung, Batch-Verarbeitung und Evaluierungs-Pipelines. Mit echtem Code in Python und TypeScript.

5. **RAG** – Du baust vollständige RAG-Pipelines: Chunking, Embeddings, Vektor-Datenbanken, Hybrid Search, Reranking und Contextual Retrieval. Du bringst deine eigenen Daten in die KI.
6. **Tool Use** – Du definierst Tools, implementierst den Tool-Use-Loop, designst sichere Tool-Schemas und baust Sicherheitsschichten. KI greift in die echte Welt.
7. **Agentische Systeme** – Du baust ReAct-Agenten, nutzt MCP, orchestriert Multi-Agent-Systeme und implementierst Guardrails. KI plant, handelt und iteriert.
8. **Fine-Tuning vs. Prompting** – Du kennst die Entscheidungsmatrix. Du weißt, dass Prompting fast immer ausreicht. Und wenn nicht: Du weißt, wie Fine-Tuning funktioniert und was es kostet.
9. **Context Engineering** – Du designst nicht nur Prompts, sondern komplette Kontexte: System-Prompt, Tools, RAG, History, Caching. Du denkst in Systemen, nicht in Einzelprompts.

Checkliste: Bin ich bereit für Band 8?

- [] Ich habe mindestens einen API-Call in Python oder TypeScript geschrieben
- [] Ich verstehe den Unterschied zwischen Chat-Interface und API-Nutzung
- [] Ich habe ein Coding-Tool (Cursor, Claude Code, Copilot) produktiv eingesetzt
- [] Ich habe eine RAG-Pipeline gebaut (oder zumindest verstanden)
- [] Ich habe Tool Use / Function Calling implementiert
- [] Ich verstehe, wann Fine-Tuning sinnvoll ist (und wann nicht)
- [] Ich kenne den Unterschied zwischen Prompt und Context Engineering
- [] Ich habe Prompt Caching verstanden und kann es einsetzen

- [] Ich habe einen einfachen Agent gebaut (oder den Ablauf verstanden)
- [] Ich kann API-Kosten für ein Projekt kalkulieren

Bei 7 von 10? Weiter zu Band 8.

Was dich in Band 8 erwartet

Band 8 heißt “Business & Produktivität” – und bringt alles Bisherige in den Arbeitsalltag.

Workflow-Automatisierung

KI in bestehende Business-Prozesse integrieren. Nicht als Spielerei, sondern als produktives Werkzeug. E-Mail-Verarbeitung, Dokumenten-Workflows, Datenaufbereitung – automatisiert.

Team-Standards für KI

Wie du KI-Nutzung im Team etablierst. Prompt-Libraries, Best Practices, Governance. Von “jeder promptet wie er will” zu “wir haben Standards”.

Berichte und Kommunikation

KI für die Textarbeit im Büroalltag: E-Mails, Berichte, Protokolle, Präsentationen. Nicht generisch, sondern in deinem Ton, für deine Zielgruppe.

ROI von KI messen

Wie du den Nutzen von KI-Integration bezifferst. Zeitersparnis, Qualitätsverbesserung, Kostensenkung – in Zahlen.

Wie die Reihe weitergeht

Anfänger (Band 1–3) ✓

- Band 1: Grundlagen ✓
- Band 2: Prompt-Frameworks ✓
- Band 3: Fortgeschrittene Basics ✓

Fortgeschritten (Band 4–6) ✓

- Band 4: Reasoning-Techniken ✓
- Band 5: Kreatives Prompting ✓
- Band 6: Spezialisiertes Prompting ✓

Profi (Band 7–9)

- Band 7: Prompting für Entwickler ✓ (*du bist hier*)
- Band 8: Business & Produktivität ← *als Nächstes*
- Band 9: Sicherheit & Ethik

Experte (Band 10)

- Band 10: Die Zukunft

Ein Gedanke zum Schluss

Dieser Band war der technischste der Reihe. Code, APIs, Vektor-Datenbanken, Agenten-Architekturen – schwerer Stoff. Aber notwendiger Stoff.

Denn die Zukunft von KI liegt nicht in Chat-Interfaces. Sie liegt in der Integration. KI als Schicht in jeder Software. KI als API-Call in jedem Workflow. KI als Agent, der Aufgaben erledigt.

Wer nur chattet, kratzt an der Oberfläche. Wer programmiert, formt die Technologie.

Und genau das hast du in diesem Band gelernt: KI formen. Nicht als passiver Nutzer, sondern als Entwickler, Architekt und Ingenieur.

In Band 8 bringen wir das in den Business-Alltag. Weniger Code, mehr Strategie. Weniger API-Calls, mehr Workflow-Design. Weniger “Wie funktioniert die Technik?” und mehr “Wie verändert die Technik meine Arbeit?”

Bis dahin: Bau etwas. Der beste Weg, Context Engineering zu lernen, ist Context Engineering zu machen.

Belkis Aslani

Ressourcen und weiterführende Links

APIs und Dokumentation:

- Anthropic API Docs: docs.anthropic.com
- OpenAI API Docs: platform.openai.com/docs
- Google AI Studio: ai.google.dev
- Claude Agent SDK: github.com/anthropics/claude-code

Coding-Tools:

- Cursor: cursor.com
- Claude Code: claude.ai/code
- GitHub Copilot: github.com/features/copilot
- Cline: github.com/cline/cline
- Aider: aider.chat

RAG und Vektor-Datenbanken:

- ChromaDB: docs.trychroma.com
- Pinecone: pinecone.io
- pgvector: github.com/pgvector/pgvector
- LangChain: python.langchain.com

MCP (Model Context Protocol):

- Spezifikation: modelcontextprotocol.io
- MCP Server Registry: github.com/modelcontextprotocol/servers

Fine-Tuning:

- OpenAI Fine-tuning Guide: platform.openai.com/docs/guides/fine-tuning
- Unsloth: github.com/unslothai/unsloth
- Hugging Face TRL: huggingface.co/docs/trl

Open Source Modelle:

- Ollama (lokale Modelle): ollama.com
- Together AI (gehostete Open Source): together.ai
- Hugging Face: huggingface.co